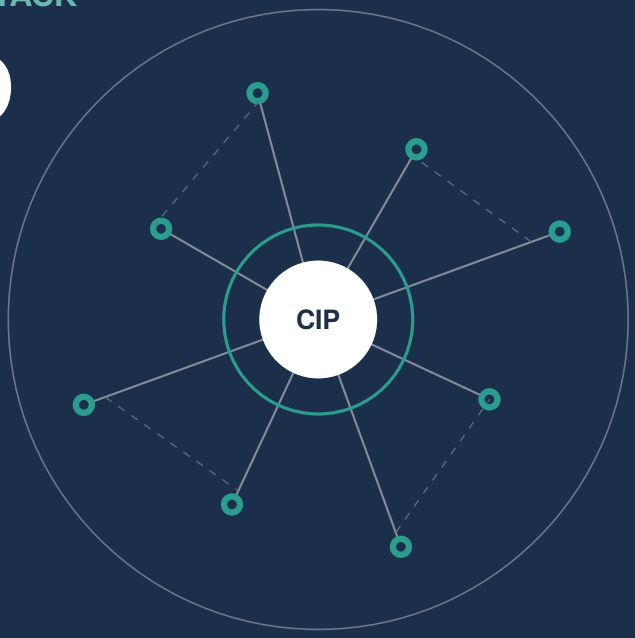
ETHERNET/IP SCANNER AND ADAPTER STACK

QuickEIP

Programming and Implementation Manual



Portable C99 for embedded targets: NetBurner, ESP32, Teknic ClearCore, Linux, and Windows

Software version 2.0.0 | QE-PIM-200, Revision A

AUTHOR AND DEVELOPER

Adam G. Sweeney

agsweeney@gmail.com

Written in ASD-STE100 Simplified Technical English.
Edited to *The Chicago Manual of Style*, 18th edition.

November 2026
MIT License

QuickEIP Programming and Implementation Manual
Document QE-PIM-200, Revision A, November 2026
For QuickEIP software version 2.0.0

Copyright © 2026 Adam G. Sweeney. All rights reserved.

The QuickEIP software is released under the MIT License. The full license text is in the file `LICENSE` at the root of the source repository. The software is supplied "as is," without warranty of any kind.

EtherNet/IP™ and CIP™ are trademarks of ODVA, Inc. ASD-STE100® is a trademark of ASD (AeroSpace, Security and Defence Industries Association of Europe). All other product names are trademarks of their respective owners. Their use in this manual identifies products and does not imply endorsement.

Status of tests. Each platform section of this manual states what has been built, what has been run, and what has been tested with a programmable logic controller (PLC). A successful build does not show that a port operates on a network. Do the hardware tests in chapter 8 before you use QuickEIP in a product.

Editorial conventions. This manual uses ASD-STE100 Simplified Technical English for its text and *The Chicago Manual of Style*, 18th edition, for its apparatus: capitalization of titles, punctuation, numbers, captions, notes, and bibliography. The preface gives the details.

Revision	Change
A, November 2026	First issue, for software version 2.0.0.

Contents

Figures vii

Tables vii

Preface xi

1 Introduction to QuickEIP 1
- 1.1 What QuickEIP Is . . . 1
- 1.2 Design Principles . . . 1
- 1.3 Scanner Functions . . . 1
- 1.4 Adapter Objects and Services . . . 1
- 1.5 Supported Platforms . . . 3
- 1.6 Limitations . . . 3

2 Architecture 5
- 2.1 Layers . . . 5
- 2.2 Source Files . . . 5
- 2.3 Execution Model . . . 7
- 2.4 Memory . . . 7

3 Building and Configuring QuickEIP 9
- 3.1 Build-Time Configuration . . . 9
- 3.2 Host Build . . . 10
- 3.3 Use QuickEIP in Your Own CMake Project . . . 12

4 Implementing an Adapter 13
- 4.1 Assemblies . . . 13
- 4.2 Start the Adapter . . . 14
- 4.3 Exchange Data . . . 15
- 4.4 Assemblies, Sizes, and the EDS File . . . 16
- 4.5 Optional Callbacks . . . 18
- 4.6 More Control: The EipServer API . . . 18

5 Connecting a PLC to the Adapter 21
- 5.1 Checks Before You Use the PLC . . . 21
- 5.2 Make the EDS File . . . 21
- 5.3 Micro850 (Connected Components Workbench) . . . 22
- 5.4 CompactLogix and ControlLogix (Studio 5000) . . . 23
- 5.5 Adapter on a PC (POSIX or Windows Port) . . . 23
- 5.6 Rules for All PLCs . . . 24

6 Using the Scanner **25**
6.1 Sessions . . . 25
6.2 Tags . . . 27
6.3 Explicit Messages . . . 27
6.4 Discovery and Symbols . . . 28
6.5 Class 1 I/O as an Originator . . . 28

7 Platform Integration **33**
7.1 NetBurner NNDK . . . 33
7.2 ESP-IDF (ESP32) . . . 35
7.3 Teknic ClearCore . . . 38
7.4 Linux, macOS, and WSL . . . 40
7.5 Windows . . . 41

8 Porting to a New Platform **43**
8.1 Select a Starting Point . . . 43
8.2 General Rules of the Contract . . . 43
8.3 Time and Scheduling . . . 44
8.4 Network Interfaces . . . 44
8.5 TCP . . . 45
8.6 UDP . . . 45
8.7 Wait and the Adapter Runner . . . 45
8.8 Check a New Port . . . 46
8.9 Rules for Changes to the Core . . . 47

9 Troubleshooting **49**
9.1 Fault Isolation Procedure . . . 49
9.2 Symptoms and Causes . . . 49
9.3 Common Mistakes . . . 50

A API Reference **53**
A.1 Utilities (`eip.h`) . . . 53
A.2 Scanner Sessions (`eip.h`) . . . 53
A.3 Discovery (`eip.h`) . . . 54
A.4 Explicit Messages and Tags (`eip.h`) . . . 55
A.5 Class 1 Originator (`eip.h`) . . . 56
A.6 Built-In Adapter (`eip.h`) . . . 57
A.7 Adapter Server (`eip_server.h`) . . . 58
A.8 Port Extras . . . 59
A.9 Port Contract (`eip_hal.h`) . . . 59
A.10 Sans-I/O Core (`eip_adapter.h`) . . . 61

B Data Structures **63**
B.1 Scanner Structures . . . 63
B.2 Adapter Structures . . . 64

C Codes and Constants **69**
C.1 Return Codes . . . 69
C.2 CIP Status Codes . . . 69
C.3 CIP Data Types . . . 69
C.4 Ports and Other Constants . . . 69
C.5 Identity Status Word of the Adapter . . . 69

D Tools, Samples, and Migration **73**
D.1 eiptool . . . 73
D.2 Examples in the Repository . . . 73
D.3 Sample EDS File . . . 73
D.4 Migration from QuickEIP 1.x . . . 75

Glossary **77**

Bibliography **79**

Function Index **81**

List of Figures

2.1 The layers of QuickEIP. Arrows show the direction of calls. The shared socket layers are reused by more than one platform port. 5

4.1 States of the consumed assembly. Red states read as all zeros. A modeless connection, which has no run/idle header, stays in "run." 16
4.2 An O→T Class 1 packet with a run/idle header. Sizes are in bytes. A T→O packet has the same items but normally no run/idle header. 16

6.1 Recovery of a lost Class 1 connection on the originator. The four calls are all necessary: a reopen of I/O on the old session fails. 30

List of Tables

1 Where to start . . . xi
2 Typographic conventions . . . xiii

1.1 Scanner functions . . . 2
1.2 CIP objects of the adapter . . . 2
1.3 Platforms and test status . . . 3

2.1 File groups in each build . . . 6
2.2 Core files and their functions . . . 6
2.3 How the adapter runs . . . 7
2.4 RAM with the default settings . . . 8

3.1 Build-time settings in `eip_config.h` . . . 10
3.2 Use of the task settings by port . . . 10
3.3 CMake targets . . . 11
3.4 Tests . . . 12

4.1 Assembly types . . . 13
4.2 The built-in test set (`config` or `assembly_count` is 0) . . . 13
4.3 Forward_Open rules and the error status for each . . . 17
4.4 Adapter callbacks . . . 18

5.1 EDS fields and the configuration values that they must agree with . . . 22
5.2 Connection values sent by a Micro850 . . . 23
5.3 Generic Ethernet Module settings . . . 23

7.1 ESP-IDF port options . . . 38
7.2 Windows port options . . . 41

8.1 Starting points for a new port . . . 43
8.2 Port functions: time, scheduling, and lock . . . 44
8.3 Port functions: TCP . . . 45
8.4 TCP options in the shipped socket layers . . . 45
8.5 Port functions: UDP . . . 46
8.6 Port functions: wait and runner . . . 46

9.1 Symptoms and their probable causes . . . 49
9.2 Common mistakes . . . 51

A.1 Sans-I/O core functions . . . 62

B.1 `EipOpenConfig`: session parameters . . . 63
B.2 `EipIoConfig`: Class 1 originator parameters . . . 63
B.3 `EipIdentity`: a ListIdentity reply . . . 64

B.4 `EipDevice`: a device found by a scan . . . 64
B.5 `EipSymbol`: one entry of the Symbol object . . . 64
B.6 `EipAssemblyConfig`: one assembly . . . 65
B.7 `EipAdapterConfig`: adapter configuration . . . 65
B.8 `EipAdapterView`: adapter snapshot . . . 66
B.9 `EipInterfaceConfig`: settable TCP/IP values . . . 67
B.10 `EipNetifInfo`: interface data from a port . . . 67

C.1 QuickEIP return codes . . . 69
C.2 CIP status codes used in this manual . . . 70
C.3 CIP elementary data types . . . 70
C.4 Network ports and constants . . . 71
C.5 Identity status of the adapter . . . 71

D.1 eiptool commands . . . 73
D.2 Example programs . . . 74
D.3 API changes from version 1.x . . . 75

Preface

Purpose of This Manual

This manual tells you how to add QuickEIP to a firmware or desktop project, how to use its programming interface, and how to connect it to a PLC. It also tells you how to port QuickEIP to a new platform.

QuickEIP is an EtherNet/IP stack in portable C99. One code base operates as a *scanner*, which sends requests to PLCs and other devices, and as an *adapter*, which a PLC sees as an I/O device.

Audience

This manual is for firmware engineers and application programmers. You must know the C language, the build system of your platform, and the basics of TCP/IP networks. Knowledge of the Common Industrial Protocol (CIP) helps, but it is not necessary. The glossary defines the CIP terms that this manual uses.

How to Use This Manual

Table 1 shows where to start for each task.

Table 1. Where to start

Task	Read
Learn what QuickEIP does and does not do	Chapter 1
Understand the layers and the threading model	Chapter 2
Build QuickEIP and set its sizes	Chapter 3
Make your device an adapter	Chapters 4 and 5
Read and write PLC tags, or open Class 1 I/O	Chapter 6
Set up NetBurner, ESP-IDF, ClearCore, Linux, or Windows	Chapter 7
Port QuickEIP to a new platform	Chapter 8
Find the cause of a fault	Chapter 9
Look up a function, a structure, or a status code	Appendixes A–C

Conventions

Language

The text of this manual follows ASD-STE100, *Simplified Technical English*.[1] STE makes the text easy to read and easy to translate. These are the rules that you will see most:

- A procedure step contains one instruction. Each step starts with a verb in the imperative, for example “Set” or “Do not.”
- A sentence in a procedure has no more than 20 words. A sentence in a description has no more than 25 words.
- The text uses the active voice and the simple tenses.
- A word has one meaning. For example, “open” always means “start a session or a connection,” and “close” always means “stop it.”
- A warning or a caution comes before the step that it applies to.

Technical names, such as function names, file names, and CIP service names, are not changed.

Style

The apparatus of this manual follows *The Chicago Manual of Style*.[2] Chapter and section titles use headline style. Lists of three or more items use the serial comma. Ranges use an en dash, for example UDP 2223–2231. The manual uses the alternative rule for numbers in Chicago 9.3: words for zero through nine in running text, and numerals for 10 and above. Numerals are used for all quantities with a unit, for data values, for instance numbers, and for code. Notes are footnotes in the Chicago notes-bibliography style, and the bibliography is at the end of the manual.

Safety Notices

△ WARNING

A warning tells you about a condition that can cause injury or death. It also tells you about a condition that can make machinery move when you do not expect it.

CAUTION

A caution tells you about a condition that can cause damage to equipment, loss of data, or corruption of memory.

NOTE: A note gives information that helps you do a task. It does not tell you about a hazard.

Typography

All timeouts in the programming interface are in milliseconds, and RPIs are in microseconds. A function that returns `int` returns `EIP_OK` (0) or a negative `EIP_ERR_*` code, unless the text gives a different rule.

[1] ASD, *ASD-STE100: Simplified Technical English*, issue 8 (Brussels: ASD, 2021).
[2] *The Chicago Manual of Style*, 18th ed. (Chicago: University of Chicago Press, 2024).

Table 2. Typographic conventions

Example	Meaning
`eip_adapter_start`	A function, a type, a macro, or a value in code.
`include/eip.h`	A file or a directory, relative to the repository root.
Project Properties	A menu, a dialog, or a field in a tool.
originator	A term that the glossary defines, at its first use.
`0x01` / `0x0127`	A CIP general status and its extended status.
O→T, T→O	The two directions of a Class 1 connection: originator to target, and target to originator.

Related Documents

The comments in the headers in `include/` are the primary reference for the programming interface. Each port with application calls has its own header: `ports/esp-idf/eip_esp_idf.h` and `ports/clearcore/eip_clearcore.h`. The repository also contains `docs/INTEGRATION.md`, `docs/PORTING.md`, and `docs/MIGRATION.md`. This manual contains the information in those files. If this manual and a header comment are different, the header comment agrees with the code. Use it, and report the difference.

The CIP and EtherNet/IP specifications are published by ODVA. The bibliography lists them.

CHAPTER 1

Introduction to QuickEIP

1.1 What QuickEIP Is

QuickEIP is a small EtherNet/IP stack for embedded targets. It is written in portable C99 and released under the MIT License. Version 2.0.0 runs on NetBurner, ESP32, Teknic ClearCore, desktop Linux, macOS, and Windows.

The stack has two roles, and one program can use both at the same time:

Scanner (originator) Your program opens a session to a PLC or a device. It sends explicit messages, reads and writes Logix tags, and opens Class 1 I/O connections.

Adapter (target) Your device answers the network as an I/O device. A PLC opens a Class 1 connection to it and exchanges assembly data at a fixed interval.

1.2 Design Principles

Three rules control the design. They make the stack small, easy to port, and easy to test.

1. **No dynamic memory in the protocol code.** All sizes are set at build time in `include/eip_config.h`. The RAM that QuickEIP uses is known before the program runs.
2. **No operating system calls in the protocol code.** A port of 19 functions supplies the clock, a lock, interface data, and non-blocking sockets. The port contract is in `include/eip_hal.h`.
3. **A sans-I/O adapter core.** The adapter core receives packets and a tick count, and it returns packets. Thus you can test it without a network, byte for byte.

1.3 Scanner Functions

Table 1.1 lists what the scanner does.

1.4 Adapter Objects and Services

Table 1.2 lists the CIP objects of the adapter.

The adapter also supplies these functions:

- ListIdentity and ListServices on TCP and UDP 44818, with a rate limit on UDP replies. ListInterfaces, RegisterSession, and SendRRData on TCP.
- Class 1 I/O on UDP 2222, with point-to-point or multicast T→O, and an optional run/idle header on O→T.
- An inactivity watchdog, checks of the sequence number and of the source address, and a defined safe state. Consumed data reads as zeros after a timeout, after a close, and in idle.
- An `allow_change` callback that can refuse an IP change or a reset, for example while machinery moves.

Table 1.1. Scanner functions

Function	Details
Sessions	Encapsulation session on TCP 44818. All explicit messages use SendRRData (unconnected).
Explicit messages	Any service with `eip_cip`, plus helpers for Get_Attribute_Single and Set_Attribute_Single. Requests up to 480 bytes.
Routing	Direct, or one hop across a ControlLogix backplane with Unconnected Send (`slot` 0–255). Tested against scripted replies, but not yet on a real ControlLogix.
Logix tags	Read and write by name, for example `Program:Main.Speed` or `Data[2,3].Value`. The scanner breaks large transfers into fragments. Data stays as raw little-endian bytes.
Symbol list	Tag list from the Symbol object, on Micro800 and Logix.
Class 1 I/O	Forward_Open as an originator, point-to-point, with an optional run/idle header. Assemblies up to 600 bytes. Your application sets the packet interval.
Discovery	ListIdentity to one address, or a broadcast scan on all interfaces.

Table 1.2. CIP objects of the adapter

CIP object	Support
Identity (`0x01`)	Attributes 1–8 and Get_Attributes_All. Reset (types 0 and 1) if the application supplies a callback.
Message Router (`0x02`)	Multiple Service Packet.
Assembly (`0x04`)	Up to eight instances by default, from a shared data pool. Get and Set of the data attribute, and of the size.
Connection Manager (`0x06`)	Forward_Open, Large_Forward_Open, and Forward_Close.
TCP/IP Interface (`0xF5`)	Read-only by default. Attributes 3, 5, and 6 become settable if the application supplies a callback.
Ethernet Link (`0xF6`)	Three read-only instances: the port in use, a second port that always reports “down,” and an internal port.

1.5 Supported Platforms

Table 1.3 lists the shipped ports and their test status. The "Verified" column tells you exactly what was done. "Builds" means that the code compiles and links. It does not mean that the port has run on a network.

Table 1.3. Platforms and test status

Platform	Port	Adapter runs in	Verified
NetBurner NNDK 3.x	`ports/netburner`	Its own task	Builds (NNDK 3.5.8, MODM7AE70). Not yet run on hardware.
ESP-IDF 5.x (ESP32)	`ports/esp-idf` + `ports/bsd_sockets`	Its own task	Builds (IDF 5.5.1, esp32). Not yet run on hardware.
Teknic ClearCore	`ports/clearcore` + `ports/lwip_raw`	`eip_adapter_poll` in the main loop	Builds and links (Enhanced ClearCore Library, arm-gcc 6.3). The adapter has run on hardware with a Micro850 PLC.
Linux, WSL, macOS	`ports/posix` + `ports/bsd_sockets`	A thread	Unit and loopback tests pass, also under ASan and UBSan (gcc 13.3, Ubuntu 24.04 on WSL2).
Windows 7 and later	`ports/windows`	A thread, or `eip_adapter_poll` with `EIP_WIN_POLL_MODE`	Unit, port-contract, and loopback tests pass (Visual Studio 2022, x64 and Win32). Explicit scanner calls checked against a Micro850. Adapter not yet tested from a real PLC.

1.6 Limitations

Compare these limits with the needs of your application before you select QuickEIP.

1.6.1 Adapter Limits

- The adapter accepts one Class 1 connection at a time. It refuses a second originator, also when T→O is multicast. Thus listen-only connections are not available.
- The adapter has no Class 3 connections. Explicit messages are unconnected only.
- The adapter has no Unconnected Send, so it cannot route.
- The T→O connection uses cyclic production. The adapter accepts a change-of-state request, but it produces on the RPI. It refuses application-triggered connections.
- A connection with a null O→T has no inactivity watchdog, and the I/O is not tied to the TCP session. The connection stays open until a Forward_Close, or until the same originator opens it again.
- An explicit reply carries no more than 508 bytes of assembly data. A Class 1 assembly has a limit of approximately 500 bytes.

1.6.2 Scanner Limits

- The scanner routes across one hop only. You can route explicit messages, but use `slot = -1` for Class 1. A routed Forward_Open has not been tested.

- The scanner does not decode tag data. Elementary types operate. `eip_tag_write` has no form for a structure, and `eip_tag_read` returns a structure with its header bytes in the data. Thus UDTs and STRINGs are not supported as types.
- Bit access by name (`Tag.3`) is not supported.
- A packet that is too short or too large closes the session, because the stream cannot be synchronized again. After a socket error or a series of timeouts, close the session and open it again. `eip_session_is_open` does not find a lost peer.

△ WARNING

QuickEIP does not implement CIP Safety. Do not use a QuickEIP connection as a safety function. Use a certified safety system to stop machinery. The safe state of consumed data (all zeros) is a functional default only. It is not a safety integrity level.

CHAPTER 2

Architecture

2.1 Layers

QuickEIP has three layers. Your application calls the public interface. The portable core contains all protocol logic. The port connects the core to one platform. Figure 2.1 shows the layers and the files in each.

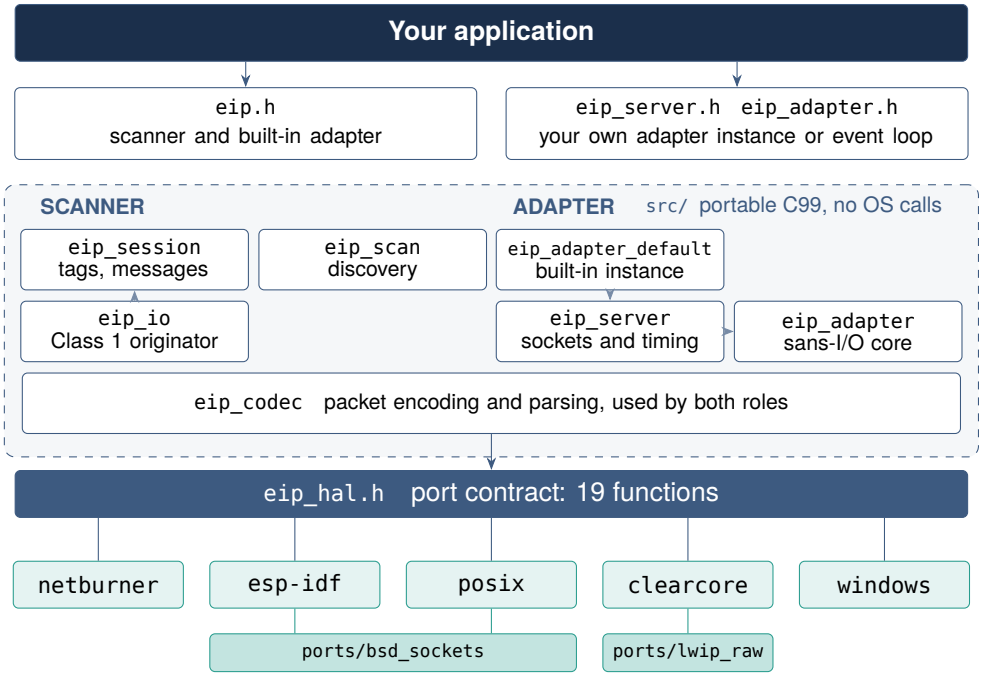


Figure 2.1. The layers of QuickEIP. Arrows show the direction of calls. The shared socket layers are reused by more than one platform port.

An application that has its own event loop can call the sans-I/O core in `include/eip_adapter.h` directly. Most applications use `eip.h` only.

2.2 Source Files

Each build compiles three groups of files: the core, one port, and the include paths. Table 2.1 lists them.

You can omit a core file if you do not use its functions. Table 2.2 shows what each file is necessary for.

> **NOTE:** If you link with `-ffunction-sections -fdata-sections -Wl,--gc-sections`, the linker removes the functions that you do not call. Thus you can usually keep all the core files in the build.

Table 2.1. File groups in each build

Group	Files	Notes
Core	`src/eip_codec.c`, `src/eip_adapter.c`, `src/eip_session.c`, `src/eip_io.c`, `src/eip_scan.c`, `src/eip_server.c`, `src/eip_adapter_default.c`	The same on all platforms. C99, no OS calls.
Port	One platform directory under `ports/`, plus a shared socket layer for some platforms	Implements `include/eip_hal.h`.
Include paths	`include/` (public) and `src/` (internal headers that the core needs)	Add both to the C and C++ compilers.

Table 2.2. Core files and their functions

File	Necessary for
`eip_codec.c`	All functions.
`eip_adapter.c`, `eip_server.c`	The adapter.
`eip_adapter_default.c`	`eip_adapter_start` and the other built-in-instance calls. Holds a static `EipServer` of approximately 11 KB.
`eip_session.c`	Scanner sessions, tags, explicit messages, and symbols.
`eip_io.c`	Scanner Class 1 I/O. Needs `eip_session.c`.
`eip_scan.c`	`eip_list_identity` and `eip_scan_devices`.

2.3 Execution Model

2.3.1 The Adapter Context

The adapter core runs in one context. On an RTOS port, this context is a task that the port starts. On a bare-metal port, this context is your main loop, which calls `eip_adapter_poll`. Table 2.3 compares the two modes.

Table 2.3. How the adapter runs

Mode	Ports	What you do
Task (RTOS)	NetBurner, ESP-IDF, POSIX, Windows	Nothing. `eip_adapter_start` starts a task. Do not call `eip_adapter_poll`.
Poll (bare metal)	ClearCore, other lwIP raw-API boards, Windows with `EIP_WIN_POLL_MODE`	Call `eip_adapter_poll` on each pass of the main loop, at intervals shorter than your fastest RPI.

CAUTION

Do not call `eip_adapter_poll` on an RTOS port. A task already runs the adapter there. Two contexts that run the core at the same time can corrupt the adapter state.

2.3.2 Rules for Tasks

- `eip_adapter_read`, `eip_adapter_write`, and `eip_adapter_snapshot` are safe from any task. They take `eip_hal_lock` for some microseconds. The lock is never held during a socket call.
- One task at a time can use a given `EipSession` or `EipIo`. Different sessions can run in different tasks.
- Scanner calls block until they are complete or until their timeout. On an RTOS, call them from a task of their own.
- On bare metal, a blocking scanner call keeps the network stack in operation, and the adapter sockets continue to receive packets. But the remainder of your main loop stops until the call returns.
- Do not call QuickEIP from an interrupt handler.

2.4 Memory

QuickEIP uses no heap in the protocol code. Its RAM is static, or it is in structures that you supply. Let P be `EIP_MAX_PACKET` and let C be `EIP_ADAPTER_MAX_CLIENTS`. Then these approximations apply:

$$M_{\text{server}} \approx P\,(C + 3), \tag{2.1}$$

$$M_{\text{session}} \approx 3P. \tag{2.2}$$

With the defaults $P = 1536$ and $C = 4$, equation (2.1) gives 10,752 bytes, and equation (2.2) gives 4,608 bytes plus some fields. Table 2.4 gives the totals.

Table 2.4. RAM with the default settings

Item	RAM
Built-in adapter	Approximately 11 KB static, plus a 1.5 KB shared reply buffer.
Each `EipSession`	Approximately 4.7 KB.
Each `EipIo`	Approximately 100 bytes, plus a 1.5 KB stack buffer during produce and consume.
Scanner calls	Up to approximately 2 KB of caller stack. A tag write builds requests of 400 bytes.
Code, all functions	Approximately 24 KB at `-Os` on a Cortex-M4.

CAUTION

An `EipSession` is too large for most task stacks. Declare it `static`, or put it in memory that you allocate one time at start-up. A session on a small stack causes a stack overflow.

The assembly data of the adapter comes from one pool. The sum of the sizes s_i of all assemblies must fit in it:

$$\sum_{i=1}^{N} s_i \le \texttt{EIP_ADAPTER_DATA_POOL}, \qquad N \le \texttt{EIP_ADAPTER_MAX_ASSEMBLIES}. \tag{2.3}$$

CHAPTER 3

Building and Configuring QuickEIP

3.1 Build-Time Configuration

All sizes are in `include/eip_config.h`. Each setting is a default in an `#ifndef` block. You can change a setting in two ways:

- Give it with `-D` on the compiler command line.
- Put it in a header of your own, and compile with `-DEIP_CONFIG_FILE=\"my_eip_config.h\"`. `eip_config.h` includes that file first.

CAUTION

All files that include `eip.h` must see the same values, in C and in C++. These settings change the size of structures. If two files see different values, they disagree about the layout of `EipSession` or `EipServer`, and memory becomes corrupt. Put the settings in a flag that applies to the C and the C++ compiler, or use `EIP_CONFIG_FILE` for both.

Procedure: Change a build-time setting

1. Make a header, for example `my_eip_config.h`.
2. Put one `#define` in it for each setting that you change:

```
/* my_eip_config.h */
#define EIP_ADAPTER_MAX_CLIENTS 2
#define EIP_ADAPTER_DATA_POOL   1024
```

3. Add `-DEIP_CONFIG_FILE=\"my_eip_config.h\"` to the C compiler flags.
4. Add the same flag to the C++ compiler flags.
5. Do a clean build of all files.

Table 3.1 lists the settings.

3.1.1 The UDP Reply Rate Limit

The adapter answers ListIdentity and ListServices on UDP 44818 through a token bucket. Each reply uses one token. Let R be `EIP_ADAPTER_UDP_REPLY_MS` and let B be `EIP_ADAPTER_UDP_REPLY_BURST`. Then, in a time t, the adapter sends no more than this number of replies:

$$n_{\max}(t) = B + \left\lfloor \frac{t}{R} \right\rfloor . \tag{3.1}$$

With the defaults, the adapter answers a burst of 20 requests and then 50 requests per second. A broadcast storm thus cannot stop Class 1 I/O. The snapshot field `udp_replies_dropped` counts the requests that the adapter did not answer.

Table 3.1. Build-time settings in `eip_config.h`

Setting	Default	Effect
`EIP_MAX_PACKET`	1536	Largest encapsulation packet. The minimum is 1024, and a smaller value stops the build. It sizes the session buffers (3×), the server reply buffers (2×), and each adapter TCP client (1×).
`EIP_ADAPTER_MAX_ASSEMBLIES`	8	Assembly instances in each adapter.
`EIP_ADAPTER_DATA_POOL`	512	Bytes that all assembly data shares.
`EIP_ADAPTER_MAX_CLIENTS`	4	Explicit-message (TCP) sessions at the same time. Each costs approximately `EIP_MAX_PACKET` bytes.
`EIP_ADAPTER_EVICT_MS`	60000	If all TCP slots are busy, the adapter drops a session that has been silent this long, for a new client. It never drops the session of the Class 1 owner.
`EIP_ADAPTER_UDP_REPLY_MS`	20	Interval at which the UDP 44818 rate limit adds one token.
`EIP_ADAPTER_UDP_REPLY_BURST`	20	Maximum number of stored tokens.
`EIP_ADAPTER_IDLE_WAIT_MS`	5	Longest wait in one call to `eip_server_service`.
`EIP_ADAPTER_TASK_STACK`	0	Adapter task stack, in the units of the RTOS. 0 selects the port default.
`EIP_ADAPTER_TASK_PRIORITY`	0	Adapter task priority, in the units of the RTOS. 0 selects the port default.

3.1.2 Task Settings

The two task settings have a meaning on some ports only. Table 3.2 shows which.

Table 3.2. Use of the task settings by port

Port	`EIP_ADAPTER_TASK_STACK`	`EIP_ADAPTER_TASK_PRIORITY`
ESP-IDF	Bytes (default 6144)	FreeRTOS priority (default 12)
NetBurner	Not used (NNDK default stack)	NBRTOS priority (default `MAIN_PRIO - 1`)
Windows	Bytes (0: system default)	A `THREAD_PRIORITY_*` value (0: `THREAD_PRIORITY_HIGHEST`)
POSIX	Not used	Not used
ClearCore	Not used (no task)	Not used (no task)

3.2 Host Build

The host build uses CMake. It needs CMake 3.13 or later and a C99 compiler. It operates on Linux, macOS, WSL, and Windows with Visual Studio.

Procedure: Build and test on Linux, macOS, or WSL

1. Go to the root of the repository.
2. Configure the build:

```
cmake -S . -B build
```

3. Compile:

```
cmake --build build
```

4. Run the tests:

```
ctest --test-dir build --output-on-failure
```

5. Make sure that all tests pass.

Procedure: Build and test on Windows with Visual Studio

1. Open a Developer Command Prompt at the root of the repository.
2. Configure the build:

```
cmake -S . -B build
```

3. Compile the Release configuration:

```
cmake --build build --config Release
```

4. Run the tests:

```
ctest --test-dir build -C Release --output-on-failure
```

5. Make sure that all tests pass.

3.2.1 Targets and Options

Table 3.3 lists the CMake targets. Two options control the optional targets: `QUICKEIP_BUILD_TESTS` and `QUICKEIP_BUILD_TOOLS`. Both are `ON` by default.

Table 3.3. CMake targets

Target	Contents
`quickeip_core`	The portable core only. Link your own `eip_hal_*` port.
`quickeip`	The core plus the host port (POSIX or Windows).
`eiptool`	The command-line tool. Set `-DQUICKEIP_BUILD_TOOLS=OFF` to omit it.
`test_core`, `test_session`, `test_hal`, `test_loopback`	The tests. Set `-DQUICKEIP_BUILD_TESTS=OFF` to omit them.

3.2.2 Tests

Table 3.4 describes each test and what it needs.

Table 3.4. Tests

Test	Covers	Needs
`test_core`	The codec and the adapter core, byte for byte. Includes Forward_Open captures from real devices.	Nothing: no sockets and no clock.
`test_session`	The scanner session against a scripted device, through a fake HAL. Routed requests, router failures, and late, short, and too-large replies.	Nothing: no sockets and no clock.
`test_hal`	The port contract, through the loopback interface of the host. Busy ports fail, long datagrams are cut, ICMP errors are not fatal, timeouts hold, and the clock steps at its stated resolution.	Free TCP and UDP ports in the 478xx range.
`test_loopback`	The scanner against the adapter through the host port. Discovery, explicit messages, Class 1 in the two directions, idle, the watchdog, and reopen.	A non-loopback interface. TCP and UDP 44818 and UDP 2222 must be free.

> **NOTE:** If another EtherNet/IP program on the host uses port 44818 or 2222, `test_loopback` fails. Stop that program, and then run the test again.

3.3 Use QuickEIP in Your Own CMake Project

Procedure: Add QuickEIP to a CMake project

1. Put the QuickEIP repository beside your project, or in a subdirectory of it.
2. Set the two build options to `OFF` before `add_subdirectory`. If you do not, the tests and `eiptool` also compile.
3. Add QuickEIP with `add_subdirectory`.
4. Link your target to `quickeip`. This gives the include paths and the thread library.

```
cmake_minimum_required(VERSION 3.13)
project(myapp C)

set(QUICKEIP_BUILD_TESTS OFF CACHE BOOL "")
set(QUICKEIP_BUILD_TOOLS OFF CACHE BOOL "")
add_subdirectory(../QuickEIP quickeip)

add_executable(myapp main.c)
target_link_libraries(myapp PRIVATE quickeip)
```

For an embedded target without CMake, compile the core files of table 2.1 and one port, and add `include/` and `src/` to the include paths. Chapter 7 gives the steps for each shipped platform.

CHAPTER 4

Implementing an Adapter

An adapter makes your device an EtherNet/IP I/O device. A PLC opens a Class 1 connection to it and exchanges assembly data at a fixed interval, the RPI. This chapter tells you how to start the adapter, exchange data, and set the assemblies so that a PLC accepts them.

4.1 Assemblies

An *assembly* is a block of bytes with an instance number. The adapter has three types of assembly. Table 4.1 gives their directions.

Table 4.1. Assembly types

Type	Direction	Meaning
`EIP_ASM_CONSUMED`	O→T	Data that the PLC writes. Your application reads it.
`EIP_ASM_PRODUCED`	T→O	Data that your application writes. The PLC reads it.
`EIP_ASM_CONFIG`	In the Forward_ Open	Configuration data. Size 0 is permitted.

CAUTION

Do not mix up the two directions. Consumed is what the PLC writes, and produced is what the PLC reads. The built-in test set uses instance 100 as consumed. The examples in this manual use instance 100 as produced.

You select the instance numbers. This manual uses 150 (consumed, 32 bytes), 100 (produced, 32 bytes), and 151 (config, 0 bytes). If you give no configuration, the adapter uses a built-in test set (table 4.2).

Table 4.2. The built-in test set (`config` or `assembly_count` is 0)

Instance	Direction	Bytes	Notes
100	Consumed	8	
101	Produced	8	Initial data `11 22 33 44 55 66 77 88`.
102	Produced	4	
150	Config	4	

With `vendor_id` 0, the adapter also uses a test identity: vendor 65000 and product name "EIP Test Adapter."

4.2 Start the Adapter

Procedure: Start the built-in adapter

1. Declare an array of `EipAssemblyConfig`, one entry for each assembly.
2. Declare an `EipAdapterConfig` and set all of it to zero with `memset`.
3. Set the identity fields: `vendor_id`, `device_type`, `product_code`, `revision_major`, `revision_minor`, and `product_name`.
4. Set `assemblies` and `assembly_count`.
5. Set the optional fields and callbacks (section 4.5).
6. Call `eip_adapter_start`.
7. Make sure that the return value is `EIP_OK`.
8. On a poll-mode port only, call `eip_adapter_poll` on each pass of the main loop.

```c
#include <string.h>
#include "eip.h"

static const EipAssemblyConfig kAssemblies[] = {
    /* instance, direction,   size, name,     initial, implicit_only */
    {150, EIP_ASM_CONSUMED, 32, "Outputs", 0, 0},   /* O->T: the PLC writes it  */
    {100, EIP_ASM_PRODUCED, 32, "Inputs",  0, 0},   /* T->O: we send it         */
    {151, EIP_ASM_CONFIG,    0, "Config",  0, 0},   /* config point of the path */
};

int start_eip(void)
{
    EipAdapterConfig cfg;

    memset(&cfg, 0, sizeof cfg);
    cfg.vendor_id = 65000;            /* placeholder: use your ODVA vendor id */
    cfg.device_type = 43;             /* 43 generic device, 12 comms adapter */
    cfg.product_code = 1;
    cfg.revision_major = 1;
    cfg.revision_minor = 1;           /* an EDS needs 1 or higher */
    cfg.product_name = "My Device";   /* up to 31 characters */
    cfg.serial_number = 0;            /* 0: derived from the MAC */
    cfg.assemblies = kAssemblies;
    cfg.assembly_count = 3;
    return eip_adapter_start(&cfg);   /* copies what it needs from cfg */
}
```

CAUTION

Set all fields of `EipAdapterConfig`, including `link_info` and the other callbacks, before you call `eip_adapter_start`. The function copies the configuration. A change after the call has no effect.

> **NOTE:** Vendor ID 65000 is a placeholder. Get your own vendor ID from ODVA, and use it in the device and in its EDS file.

`eip_adapter_start` copies what it needs, so `cfg` and the assembly array can be on the stack. The adapter listens on TCP 44818, UDP 44818, and UDP 2222. If the network is not ready, the adapter tries again to open its sockets one time each second. The field `listening` in `EipAdapterView` becomes 1 when the sockets are open.

4.3 Exchange Data

Use `eip_adapter_read` to get the data that the PLC sent. Use `eip_adapter_write` to set the data that the PLC receives.

```
uint8_t out[32], in[32];
uint16_t len = sizeof out;

eip_adapter_read(150, out, &len);   /* latest data from the PLC          */
/* ... use out, fill in ... */
eip_adapter_write(100, in, 32);     /* sent on the next T->O packet      */
```

These rules apply:

- The two calls are safe from any task.
- `eip_adapter_write` needs the exact size of the assembly.
- For `eip_adapter_read`, `data_len` is the capacity on input and the byte count on output. A capacity smaller than the assembly is `EIP_ERR_TOO_BIG`.
- An unknown instance returns `EIP_ERR_PATH`.

4.3.1 The Safe State of Consumed Data

The consumed assembly reads as all zeros in these conditions:

- The connection timed out or closed.
- The PLC is in Program mode. The run/idle header then says “idle.”

Before the first connection, the consumed assembly reads as its `initial` data. That data is also zeros if you do not set it. Figure 4.1 shows the states.

> **△ WARNING**
>
> Design your application so that all-zero consumed data puts your machine in a safe condition. When the connection stops, or when the PLC goes to Program mode, your outputs see zeros. If zero is not a safe value for an output, add logic that makes it safe. Do not use this function as a safety function.

4.3.2 Status and Diagnostics

`eip_adapter_snapshot` copies the adapter state into an `EipAdapterView`. It gives the identity, the connection state, the RPI, the inactivity timeout, and these counters:

- packets in and out, and successful opens and timeouts

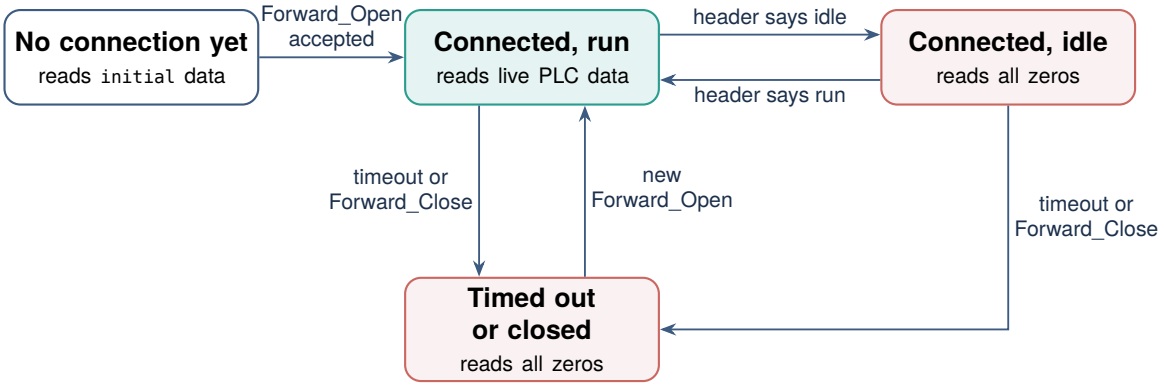


Figure 4.1. States of the consumed assembly. Red states read as all zeros. A modeless connection, which has no run/idle header, stays in "run."

- drops by cause: old sequence number (`rx_dropped_seq`), wrong source address (`rx_dropped_source`), and malformed packet (`rx_dropped_bad`)
- packets accepted in idle (`rx_idle`), and UDP replies not sent (`udp_replies_dropped`)

Use these values on a status page. Appendix B lists all fields. The Identity object status word also shows the state: `0x0034` with no connection, `0x0065` in run, and `0x0075` in idle.

4.4 Assemblies, Sizes, and the EDS File

The PLC sets up the connection from your EDS file. The adapter refuses a Forward_Open that does not agree with its assemblies.

4.4.1 Assembly Size and Wire Size

In an EDS file and in PLC software, you enter the *assembly* size. The 2-byte sequence count and the 4-byte run/idle header are added on the wire. The adapter checks the wire size. Let n be the assembly size in bytes, and let h be 1 if the direction has a run/idle header, else 0. Then:

$$S_{\mathrm{O}\to\mathrm{T}} = 2 + 4h + n, \tag{4.1}$$

$$S_{\mathrm{T}\to\mathrm{O}} = 2 + n. \tag{4.2}$$

For a 32-byte assembly in each direction, $S_{\mathrm{O}\to\mathrm{T}} = 38$ and $S_{\mathrm{T}\to\mathrm{O}} = 34$. Figure 4.2 shows where these bytes are in an O→T packet.

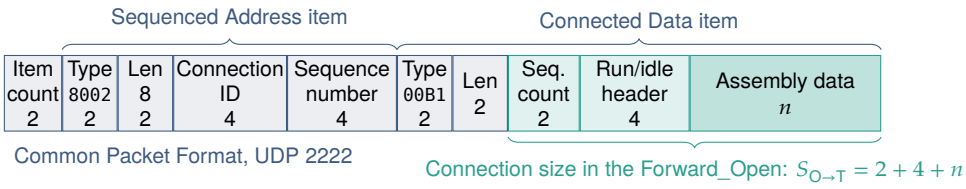


Figure 4.2. An O→T Class 1 packet with a run/idle header. Sizes are in bytes. A T→O packet has the same items but normally no run/idle header.

The run/idle header is a format flag in the EDS file. It is not part of the size that you enter.

4.4.2 Rules for a Forward_Open

Table 4.3 gives the rules that the adapter applies to a Forward_Open, and the status that it returns when a rule is not met. The sizes in this table are wire sizes.

Table 4.3. Forward_Open rules and the error status for each

Rule	Status if not met
The connection path names an existing consumed, produced, and config instance.	`0x01` / `0x012A` (consumed), `0x012B` (produced), `0x0129` (config)
O→T size = $2 + 4h + n$ (equation 4.1).	`0x01` / `0x0127`
T→O size = $2 + n$. If the size also includes a 4-byte header, the adapter accepts it and sends a header that always says "run."	`0x01` / `0x0128`
Config data in the path, if any, is equal to the config assembly size.	`0x01` / `0x0126`
The electronic key, if the PLC checks it, agrees with the vendor, device type, product code, and revision. A zero in the key agrees with all values.	`0x01` / `0x0114–0x0116`
No other originator owns the connection.	`0x01` / `0x0106`, or `0x01` / `0x0113` if the request has no O→T

A size-0 `EIP_ASM_CONFIG` assembly accepts a path with a config point and no data. Most PLCs send that. Set `implicit_only` to 1 to hide an assembly from explicit Get and Set (status `0x16`). Class 1 can still use that assembly.

4.4.3 The Inactivity Timeout

The adapter closes the connection if no valid O→T packet arrives in the inactivity timeout. The timeout comes from the O→T RPI and the timeout multiplier m (0–7) in the Forward_Open.[1] Let T_f be `timeout_floor_ms` and let Δ be the clock resolution of the port:

$$T_{\text{inactivity}} = \max(\text{RPI}_{\text{O}\to\text{T}} \cdot 4 \cdot 2^m,\ T_f,\ 2\Delta)\,. \tag{4.3}$$

The result is rounded up to whole ticks. In ticks, it is also limited to `0x00FFFFFF`. For example, a Micro850 sends an RPI of 50 ms with $m = 0$, so $T_{\text{inactivity}} = 50 \times 4 = 200$ ms. A multiplier of 2 with a 10 ms RPI gives $10 \times 16 = 160$ ms.

> **NOTE:** On a platform that is not real time, such as Windows, set `timeout_floor_ms` to 500 or more. A 100 ms stall of the host then does not close the connection.

4.4.4 Multicast T→O

If the PLC requests a multicast T→O connection, the adapter selects the group address from its own IP address a and its network mask k, as the EtherNet/IP specification tells.[2] Let the host ID be $h = a \wedge \neg k$. Then:

$$A_{\text{mcast}} = 239.192.1.0 + 32 \cdot ((h - 1) \wedge \texttt{0x3FF}). \tag{4.4}$$

Multicast does not let a second originator listen. The adapter still accepts one Class 1 connection only.

[1] ODVA, *The CIP Networks Library*, vol. 1, *Common Industrial Protocol*, chap. 3, Connection Manager object.
[2] ODVA, *The CIP Networks Library*, vol. 2, *EtherNet/IP Adaptation of CIP*, sec. 3-5.3.

4.5 Optional Callbacks

Table 4.4 lists the callbacks in `EipAdapterConfig`. All of them run in the adapter context, the task or `eip_adapter_poll`. Each receives `callback_ctx` as its first argument.

Table 4.4. Adapter callbacks

Field	When it is called	Use
`link_info`	Each 100 ms	Gives link state, speed, and duplex for the Ethernet Link object, if the port cannot read them (ESP-IDF). Return 1 while the link is up, and 0 to report it down.
`apply_interface`	Approximately 150 ms after a scanner sets TCP/IP Interface attribute 3, 5, or 6. The reply has already gone out.	Store and apply the new IP settings. Set it to 0 to make those attributes read-only.
`reset`	Approximately 300 ms after an Identity Reset request	Restart (type 0), or set the factory settings and restart (type 1). Set it to 0 to refuse the service.
`allow_change`	Before the adapter accepts a TCP/IP Interface Set or an Identity Reset	Return 0 to permit the change. Return a CIP general status, usually `0x10` (device state conflict), to refuse it.

⚠ WARNING

A scanner on the network can send an Identity Reset or change the IP address of your device. If your device controls machinery, supply `allow_change`. Make it refuse these requests while the machinery can move.

CAUTION

`allow_change` runs while the adapter holds `eip_hal_lock`. Keep it short. Do not call an `eip_adapter_*` function from it, because that function also takes the lock.

A request that `allow_change` refuses never gets to `apply_interface` or `reset`.

4.6 More Control: The EipServer API

The functions `eip_adapter_*` use one built-in instance of `EipServer`. The header `eip_server.h` gives access to that type. Use it for these tasks:

- Put the 11 KB server in memory that you select.
- Run the loop from your own task.
- Stop the adapter and start it again.

```
static EipServer server;

eip_server_init(&server, &config);
for (;;) {
    eip_server_service(&server, 5);   /* RTOS task: waits up to 5 ms */
}
```

```
/* bare metal: call eip_server_poll(&server) from the main loop instead */
```

`eip_server_init`, `eip_server_poll`, `eip_server_service`, and `eip_server_close` run in one context. `eip_server_read`, `eip_server_write`, and `eip_server_snapshot` are safe from any task, because they take `eip_hal_lock`.

4.6.1 The Sans-I/O Core

The header `eip_adapter.h` gives the core under the server. The core never uses a socket or a clock. You give it packets and a tick count, and you send the packets that it returns. Use it only if you write your own event loop. The core has one shared reply buffer. Thus you must serialize the calls on all `EipAdapter` objects in a program. Appendix A lists its functions.

CHAPTER 5

Connecting a PLC to the Adapter

This chapter tells you how to make an EDS file for your adapter and how to connect a PLC to it. Do the checks in section 5.1 before you start the PLC software.

5.1 Checks Before You Use the PLC

Procedure: Check the adapter from a PC

1. Connect a PC to the same network as the adapter.
2. Build `eiptool` on the PC (section 3.2).
3. Read the identity of the adapter:

```
eiptool identity <adapter-ip>
```

4. Make sure that the vendor, product code, and name are correct.
5. Read the data of the produced assembly:

```
eiptool get <adapter-ip> 4 100 3
```

6. Make sure that the byte count is equal to the assembly size.

5.2 Make the EDS File

The file `examples/eds/QuickEIP_Adapter.eds` agrees with the assemblies in section 4.2. Use it as the start for your EDS file. Table 5.1 shows which values must agree with your configuration.

For the example assemblies, config 151 is `0x97`, consumed 150 is `0x96`, and produced 100 is `0x64`. Thus the connection path is `20 04 24 97 2C 96 2C 64`.

Procedure: Adapt the sample EDS file

1. Copy `examples/eds/QuickEIP_Adapter.eds` and give the copy a new name.
2. Set the `[Device]` values from table 5.1.
3. Set each `Assem`*NNN* entry to agree with one assembly.
4. Set the connection sizes and `TSpec1` and `TSpec2` to the assembly sizes.
5. Set the connection path to your config, consumed, and produced instances.
6. Do not change the other values.
7. Do the checks of section 5.1 again.

Table 5.1. EDS fields and the configuration values that they must agree with

EDS field	Must be equal to
`VendCode`, `ProdType`, `ProdCode`	`vendor_id`, `device_type`, `product_code`
`MajRev`, `MinRev`	`revision_major`, `revision_minor`. EDS tools need 1–255, so do not use a minor revision of 0.
`ProdName`	`product_name`
`VendName`, `Catalog`	Your company and your part number. No configuration field agrees with these.
`Assem`*NNN*, its size, and its member size in bits	An `EipAssemblyConfig` instance, its `size`, and `size` × 8
Connection sizes, `TSpec1`, `TSpec2`	The assembly sizes (not the wire sizes)
`Number_Of_Static_Instances` under `[Assembly]`	`assembly_count`
Connection path	`20 04 24` *config* `2C` *consumed* `2C` *produced*, with instances in hexadecimal, for instances up to 255
Assembly path	`20 04 24` *instance* `30 03`, with the instance in hexadecimal

CAUTION

When an assembly layout changes, also change `ProdCode` or the revision. If you do not, a PLC can use an old EDS file with the new device, and the data has a different meaning.

NOTE: The connection values in the sample file were checked: they opened a connection to a running adapter. But nobody has imported the file itself into PLC software yet.

Appendix D contains the important parts of the sample file.

5.3 Micro850 (Connected Components Workbench)

This setup was tested with the ClearCore port.

Procedure: Connect a Micro850

1. In Connected Components Workbench, add a generic EtherNet/IP device.
2. Set its address to the address of the adapter.
3. Set the output assembly (O→T) to instance 150, 32 bytes.
4. Set the input assembly (T→O) to instance 100, 32 bytes.
5. Set the config assembly to instance 151, 0 bytes.
6. Download the project to the controller.
7. Put the controller in Run mode.
8. On the adapter, make sure that `io_open` and `run` are 1 in the snapshot.

Table 5.2 shows the values that a Micro850 was seen to send.
The config instance must exist. If it does not, the adapter refuses the open with `0x01` / `0x0129`.

Table 5.2. Connection values sent by a Micro850

Setting	Value
RPI	50 ms in each direction
O→T	Point-to-point, with the run/idle header
T→O	No header
Timeout multiplier	0 (×4, thus 200 ms at a 50 ms RPI)
Electronic key	All zeros
Config	Names the config instance and sends no data

A size-0 `EIP_ASM_CONFIG` assembly is sufficient.

5.4 CompactLogix and ControlLogix (Studio 5000)

This adapter has not been tested with these controllers. But an EDS file with the same connection values, and generic module settings in this form, have operated on a CompactLogix with a different adapter.

You can install the EDS file with the EDS Hardware Installation Tool and add the device as a module. Or you can add a Generic Ethernet Module with the values in table 5.3.

Table 5.3. Generic Ethernet Module settings

Field	Value
Comm Format	Data – SINT
IP Address	The address of the adapter
Input	Assembly instance 100, size 32
Output	Assembly instance 150, size 32
Configuration	Assembly instance 151, size 0

Then set the RPI on the Connection tab. In the two methods, the config assembly must exist on the adapter, also at size 0. It comes first in the connection path.

5.5 Adapter on a PC (POSIX or Windows Port)

This setup has not been tested with a PLC.

Procedure: Prepare a PC adapter for a PLC

1. In the firewall, open TCP 44818, UDP 44818, and UDP 2222.
2. If the PC has more than one interface, set `EIP_NETIF` to the interface that the PLC uses.
3. Under WSL2, use mirrored networking.
4. Set the RPI in the PLC to 50 ms or more.
5. Set `timeout_floor_ms` to 500 or more.

A PC has less regular timing than an embedded target. Thus start with a longer RPI.

5.6 Rules for All PLCs

- The adapter accepts one Class 1 connection at a time. It refuses a second originator.
- Keep the RPI much longer than the clock step of your port. On a NetBurner at 20 Hz, use 50 ms or more (section 7.1).
- If the PLC shows a fault, find the status in table 4.3. `16#0127` and `16#0128` are size errors.
- Outputs read as zeros while the PLC is in Program mode.

CHAPTER 6

Using the Scanner

The scanner lets your program send requests to a PLC or a device. All scanner calls block until they are complete or until their timeout. On an RTOS, call them from a task of their own (section 2.3).

6.1 Sessions

A *session* is a TCP connection to port 44818 with a registered session handle. All explicit messages and the Forward_Open of a Class 1 connection go through a session.

6.1.1 The Slot

The `slot` field of `EipOpenConfig` tells the scanner how to send each request:

- `slot = -1` sends the request directly to the device. Use it for CompactLogix, Micro800, and all devices that you reach directly.
- `slot` 0–255 puts the request in an Unconnected Send. The request goes across ControlLogix backplane port 1 to that slot.

CAUTION

A zeroed `EipOpenConfig` has slot 0, which routes across a backplane. For a device that you reach directly, set `slot = -1`. If you do not, the device can refuse the requests, or a router can send them to the wrong module.

6.1.2 Open and Close a Session

Procedure: Open a session and read a tag

1. Declare the `EipSession` as `static`. It is approximately 4.7 KB.
2. Set an `EipOpenConfig` to zero.
3. Set `ip` to the address of the device, as dotted text.
4. Set `slot`. Use −1 for a device that you reach directly.
5. Optional: set `timeout_ms`. The value 0 selects 5000 ms.
6. Call `eip_session_open`.
7. If the result is not `EIP_OK`, stop. Use `eip_status_text` to show the cause.
8. Send your requests.
9. Call `eip_session_close`.

```c
#include <stdio.h>
#include "eip.h"

static EipSession s;   /* about 4.7 KB: keep it off the stack */

int read_speed(void)
{
    EipOpenConfig open = {0};
    uint8_t buf[4];
    uint16_t len = sizeof buf, type = 0;
    uint8_t status = 0;
    int rc;

    open.ip = "192.168.1.10";
    open.slot = -1;           /* direct. A zeroed config means slot 0! */
    open.timeout_ms = 2000;   /* 0 selects 5000 */
    open.vendor_id = 65000;   /* used only by a Class 1 Forward_Open */

    rc = eip_session_open(&s, &open);
    if (rc != EIP_OK) {
        printf("open failed: %s\n", eip_status_text(rc));
        return rc;
    }
    rc = eip_tag_read(&s, "Program:Main.Speed", 1, buf, &len, &type, &status);
    if (rc == EIP_OK && type == EIP_DINT) {
        printf("Speed = %d\n", (int)(int32_t)eip_load_le32(buf));
    } else if (rc == EIP_ERR_CIP) {
        printf("CIP status 0x%02X / 0x%04X\n",
               eip_last_cip_status(&s), eip_last_cip_extended(&s));
    }
    eip_session_close(&s);
    return rc;
}
```

6.1.3 Rules for Sessions

- `eip_session_open` writes over the session. Close a session before you open it again.
- `eip_session_is_open` is safe on a zeroed session that you never opened.
- After `EIP_ERR_TIMEOUT`, the session usually stays open. The next request discards a late reply.
- A packet that is too short or too large closes the session, because the stream cannot be synchronized again. The call that failed returns its error. Later calls return `EIP_ERR_STATE` until you open the session again.
- `eip_session_is_open` tells you only that the session is registered here. It stays 1 after the peer stops.

CAUTION

Do not use `eip_session_is_open` to find a lost device. After `EIP_ERR_IO`, or when requests continue to time out, close the session and open it again.

6.2 Tags

`eip_tag_read` and `eip_tag_write` access Logix tags by name.

- Use a period for a member and `[n]` or `[n,m]` for an element: `Program:Main.Speed`, `Data[2,3].Value`.
- The scanner breaks reads and writes that are larger than one packet into fragments (Read Tag Fragmented and Write Tag Fragmented).
- Data is raw little-endian bytes. Use the helpers `eip_load_le16`, `eip_load_le32`, `eip_store_le16`, and `eip_store_le32`.
- `cip_type` receives the CIP type code of the tag, for example `EIP_DINT` (`0x00C4`). Table C.3 lists the codes.

```
uint8_t buf[4];
uint8_t status = 0;

eip_store_le32(buf, 1234);
eip_tag_write(&s, "Setpoint", EIP_DINT, 1, buf, 4, &status);
```

CAUTION

A write of more than one fragment is not atomic. If an error occurs during the write, the PLC can keep the fragments that it already received. Thus the tag can contain a mix of old and new data. Do not write large arrays that a running PLC program uses for control while the write is in progress.

6.3 Explicit Messages

`eip_cip` sends one CIP service with a raw path. Use it for a service that has no helper. The helpers `eip_get_attribute` and `eip_set_attribute` send Get_Attribute_Single and Set_Attribute_Single.

```
uint8_t buf[4];
uint16_t len = sizeof buf;
uint8_t status = 0;

/* Identity object (class 0x01), instance 1, attribute 1: vendor ID */
eip_get_attribute(&s, 0x01, 1, 1, buf, &len, &status);
```

- The encoded request has a limit of 480 bytes.
- A general status other than 0 gives `EIP_ERR_CIP`. The exception is `0x06` (partial transfer), which gives `EIP_OK` with the status in `*cip_status`.
- When a call returns `EIP_ERR_CIP`, `eip_last_cip_status` gives the general status and `eip_last_cip_extended` gives the extended status.
- On a routed session (slot 0 or more), the router can cause the error. For example, `0x01` / `0x0204` means that the slot did not answer.

6.4 Discovery and Symbols

```
EipDevice found[16];
uint16_t n = 16;
eip_scan_devices(found, &n, 0);       /* broadcast on each interface */

EipIdentity id;
eip_list_identity("192.168.1.10", &id, 0);

static EipSymbol tags[256];
uint16_t count = 256;
eip_list_symbols(&s, tags, &count);   /* Micro800: open with slot -1 */
```

- `eip_scan_devices` sends a broadcast ListIdentity on each interface and on the global broadcast address. With a timeout of 0, it waits approximately 3 s, sends again each second, and stops 0.5 s after the last reply. `EIP_OK` with a count of 0 means that no device answered.
- Broadcasts do not cross routers or VLANs. For a device on a different subnet, use `eip_list_identity` with its address.
- `eip_list_symbols` reads the Symbol object. If a later page fails, the count still tells you how many symbols the call stored.
- `eip_device_kind` sorts Rockwell products into Micro800, CompactLogix, or other by the catalog name. This is a heuristic. Your application decides what to do with the result.

6.5 Class 1 I/O as an Originator

The scanner can open a Class 1 connection to an adapter. Your program then produces O→T data and consumes T→O data.

Procedure: Open a Class 1 connection

1. Open a session with `slot = -1` and your vendor ID in `vendor_id`.
2. Set an `EipIoConfig` to zero.
3. Set `config_instance` to the value in the EDS file of the target.
4. Set `output_instance` and `output_bytes` for the data that you produce.
5. Set `input_instance` and `input_bytes` for the data that you consume.
6. Set `o2t_rpi_us` and `t2o_rpi_us`.
7. Set `run_idle` to 1 if the target expects the run/idle header.
8. Set `timeout_multiplier`.
9. Call `eip_io_open`.
10. Make sure that the result is `EIP_OK`.

```
static EipIo io;
EipIoConfig ioc = {0};
```

```
uint8_t out[32] = {0}, in[32];

ioc.config_instance = 151;
ioc.output_instance = 150;  ioc.output_bytes = 32;   /* we produce (O->T) */
ioc.input_instance  = 100;  ioc.input_bytes  = 32;   /* we consume (T->O) */
ioc.o2t_rpi_us = ioc.t2o_rpi_us = 10000;             /* 10 ms */
ioc.run_idle = 1;
ioc.timeout_multiplier = 2;                          /* timeout = RPI x 16 */

if (eip_io_open(&s, &io, &ioc) == EIP_OK) {
    for (;;) {                                       /* once per RPI */
        uint16_t in_len = sizeof in;
        eip_io_produce(&io, out, 32, 1 /* run */);
        eip_io_consume(&io, in, &in_len, 100);
    }
    eip_io_close(&s, &io);
}
```

These rules apply:

- The originator binds UDP 2223–2231, unless you set `local_port`. Thus it can share a device with an adapter on 2222.
- The library does not set the packet interval. Call `eip_io_produce` one time each RPI.
- `eip_io_consume` waits for a packet. A loop like the example thus runs at the T→O rate. A timeout of 0 means 1 s. There is no non-blocking form. Use a short timeout, or use different tasks for produce and consume.
- `config_instance = 0` sends instance 0 in the path. The QuickEIP adapter accepts it, but a different target can refuse it.
- The connection size sent to the target follows equation (4.1).

CAUTION

Use the address that the adapter reports, which is the address that a scan shows. The adapter tells the originator to send to that address, and it accepts O→T data only from the address of the Forward_Open. If you use a different address of the same host, for example `127.0.0.1`, the adapter drops the packets and the connection times out. Explicit messages are not affected.

NOTE: Open Class 1 on a session with `slot = -1`. On a routed session, the Forward_Open goes through Unconnected Send. That combination has not been tested.

6.5.1 Recover a Lost Connection

`eip_io_produce` uses UDP, so it continues to succeed after the target stops. A lost connection shows only in `eip_io_consume`, which returns `EIP_ERR_TIMEOUT` when no T→O packet arrives. Figure 6.1 shows the recovery sequence.

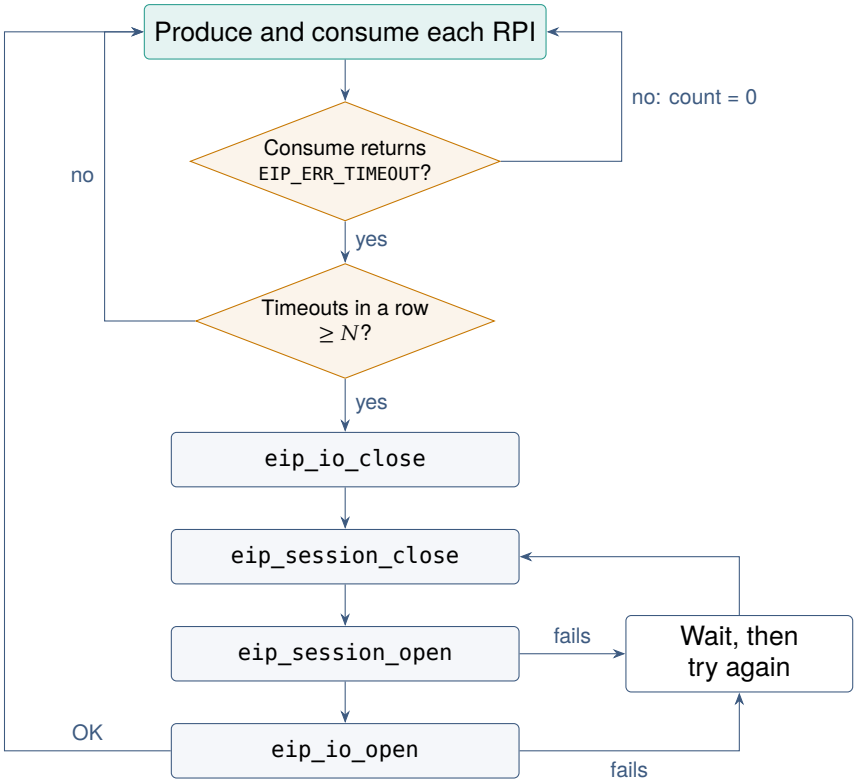


Figure 6.1. Recovery of a lost Class 1 connection on the originator. The four calls are all necessary: a reopen of I/O on the old session fails.

Procedure: Recover a lost Class 1 connection

1. Count the calls to `eip_io_consume` in a row that return `EIP_ERR_TIMEOUT`.
2. When the count gets to your limit, call `eip_io_close`.
3. Call `eip_session_close`.
4. Call `eip_session_open`.
5. If the open fails, wait and do step 3 again.
6. Call `eip_io_open`.
7. Set the count to 0, and start the produce and consume loop again.

NOTE: When the peer is gone, `eip_io_close` can block for the session timeout and return an error. This is the expected result. The call still releases the UDP socket.

If you do not close and open the session, `eip_io_open` returns `EIP_ERR_IO`.

CHAPTER 7

Platform Integration

This chapter gives the steps to add QuickEIP to a project on each shipped platform. Table 1.3 gives the test status of each port.

7.1 NetBurner NNDK

The port was tested with NNDK 3.5.8 on ARM platforms. The NNDK 3.x socket API is the same on all platforms.

Procedure: Add QuickEIP to an NNDK project

1. Put QuickEIP in a subdirectory of the project, for example `MyProject/QuickEIP`. A git submodule is also correct.
2. Write the makefile of the project (listing below).
3. Set the NNDK environment variables.
4. Run `make`.
5. Find the image at `obj/release/MyProject.bin`.

> **NOTE:** The NNDK rules put each object file at `obj/release/`*source path*. A source path that starts with `../` thus puts the object files outside `obj/`, next to your sources. Keep QuickEIP inside the project directory to prevent this.

This is a complete makefile, with QuickEIP in a subdirectory:

```
NAME    = MyProject

CPP_SRC += src/main.cpp

QUICKEIP = QuickEIP

C_SRC   += $(QUICKEIP)/src/eip_codec.c \
           $(QUICKEIP)/src/eip_adapter.c \
           $(QUICKEIP)/src/eip_session.c \
           $(QUICKEIP)/src/eip_io.c \
           $(QUICKEIP)/src/eip_scan.c \
           $(QUICKEIP)/src/eip_server.c \
           $(QUICKEIP)/src/eip_adapter_default.c
CPP_SRC += $(QUICKEIP)/ports/netburner/eip_hal_netburner.cpp
NBINCLUDE += -I$(QUICKEIP)/include -I$(QUICKEIP)/src

include $(NNDK_ROOT)/make/boilerplate.mk
```

Set the environment and build, for example:

```
export NNDK_ROOT=C:/nburn PLATFORM=MODM7AE70 DEFPLATFORM=MODM7AE70
```

```
export PATH=/c/nburn/gcc/bin:/c/nburn/pcbin:/c/nburn/gcc/msys/1.0/bin:$PATH
make
```

The first build takes several minutes, because the NNDK compiles its system library into the `obj/` directory of the project. This is normal. For NBEclipse, add the same files to the project. Add the two include directories to the C and the C++ compiler settings.

7.1.1 Start QuickEIP from UserMain

Scanner calls block, so run them in a task of their own. Each task needs its own priority, and a lower number runs first. The adapter task uses `MAIN_PRIO - 1`. Thus `MAIN_PRIO + 1` is a safe priority for the scanner task. `UserMain` must then sleep, so that the scanner task can run.

```
#include <init.h>
#include <nbrtos.h>
#include <string.h>

#include "eip.h"

const char *AppName = "MyProject";

static void ScannerTask(void *)
{
    for (;;) {
        /* eip_session_open, eip_tag_read, and so on (chapter 6) */
        OSTimeDly(TICKS_PER_SECOND);
    }
}

void UserMain(void *)
{
    EipAdapterConfig cfg;

    init();
    WaitForActiveNetwork(TICKS_PER_SECOND * 10);

    memset(&cfg, 0, sizeof cfg);
    /* fill in cfg as in chapter 4 */
    eip_adapter_start(&cfg);
    OSSimpleTaskCreatewName(ScannerTask, MAIN_PRIO + 1, "EipScanner");

    for (;;) {
        /* eip_adapter_read and eip_adapter_write */
        OSTimeDly(1);
    }
}
```

7.1.2 Platform Notes

Clock resolution. `eip_hal_millis` follows `TimeTick`: 50 ms at the default 20 Hz. The adapter never sets a Class 1 timeout shorter than two ticks (100 ms), and T→O packets go out on tick boundaries. For RPIs shorter than approximately 50 ms, increase `TICKS_PER_SECOND` in the platform configuration.

Task. The adapter task runs at `MAIN_PRIO - 1` by default. Set `EIP_ADAPTER_TASK_PRIORITY` to change it. The task uses the default NNDK task stack.

Lock. `eip_hal_lock` is `OSLock()`, which stops task switches for a short time. It is never held during a socket call.

UDP sockets. The port uses `CreateRxTxUdpSocket4` with an explicit local port. When the library asks for "any port," the port selects one in 50000–50063.

TCP options. The port does not set TCP_NODELAY or keep-alive. A slot of a lost peer comes back through eviction after `EIP_ADAPTER_EVICT_MS`.

Web UI. `examples/netburner_webui` is a test web page. It is application code, not part of the stack. It has no `main` and no makefile, so add it to an NNDK application.

7.2 ESP-IDF (ESP32)

The port was tested with ESP-IDF 5.5.1 on the esp32 target. All ESP-IDF 5.x versions should operate.

7.2.1 Add the Component

The root directory of QuickEIP is an IDF component. The component takes the name of its directory. Use the same spelling in all locations. The simpler method is a top-level `CMakeLists.txt` that names the directory. The path is relative to the project. Here QuickEIP is beside it:

```
cmake_minimum_required(VERSION 3.16)
list(APPEND EXTRA_COMPONENT_DIRS "${CMAKE_CURRENT_LIST_DIR}/../QuickEIP")
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(myapp)
```

With the component manager, put the dependency in `main/idf_component.yml`. The path is relative to `main/`:

```
dependencies:
  QuickEIP:
    path: ../../QuickEIP
```

CAUTION

The dependency key must be the same as the directory name. If it is not, the build fails with `Failed to resolve component ... unknown name`. A change to `REQUIRES` does not correct this error.

With the two methods, `main/CMakeLists.txt` names the component and the network components that you use:

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS "."
                       REQUIRES QuickEIP esp_eth esp_netif esp_event)
```

Build with `idf.py set-target esp32` and then `idf.py build`.

7.2.2 Start Ethernet

Use the driver code in `examples/ethernet/basic` of ESP-IDF as a model.

> **⚠ WARNING**
>
> You must know the PHY, the pins, and the RMII clock mode of your board. A wrong value compiles without an error, but the link never comes up. Get these values from the schematic of the board. Do not guess them.

This example is for the internal EMAC of the ESP32 with a LAN87xx PHY:

```
#include "esp_eth.h"
#include "esp_event.h"
#include "esp_netif.h"

static esp_eth_handle_t s_eth;
static esp_netif_t *s_netif;
static volatile int s_link_up;

static void eth_event(void *arg, esp_event_base_t base, int32_t id, void *data)
{
    (void)arg; (void)base; (void)data;
    if (id == ETHERNET_EVENT_CONNECTED) s_link_up = 1;
    else if (id == ETHERNET_EVENT_DISCONNECTED) s_link_up = 0;
}

static void eth_start(void)
{
    eth_mac_config_t mac_cfg = ETH_MAC_DEFAULT_CONFIG();
    eth_esp32_emac_config_t emac_cfg = ETH_ESP32_EMAC_DEFAULT_CONFIG(); /* set smi_gpio
    and clock_config */
    eth_phy_config_t phy_cfg = ETH_PHY_DEFAULT_CONFIG();                /* and your PHY
    address */
    esp_eth_mac_t *mac = esp_eth_mac_new_esp32(&emac_cfg, &mac_cfg);
    esp_eth_phy_t *phy = esp_eth_phy_new_lan87xx(&phy_cfg);
    esp_eth_config_t eth_cfg = ETH_DEFAULT_CONFIG(mac, phy);
    esp_netif_config_t netif_cfg = ESP_NETIF_DEFAULT_ETH();

    ESP_ERROR_CHECK(esp_eth_driver_install(&eth_cfg, &s_eth));
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    s_netif = esp_netif_new(&netif_cfg);
    ESP_ERROR_CHECK(esp_netif_attach(s_netif, esp_eth_new_netif_glue(s_eth)));
    ESP_ERROR_CHECK(esp_event_handler_register(ETH_EVENT, ESP_EVENT_ANY_ID, eth_event,
    NULL));
    ESP_ERROR_CHECK(esp_eth_start(s_eth));
}
```

7.2.3 Report the Link

The esp_netif layer does not report speed or duplex. Read them from the driver in a `link_info` callback. Return 1 while the link is up, and 0 to report it down:

```
static int link_info(void *ctx, uint32_t *mbps, int *full, int *autoneg)
```

```
{
    eth_speed_t speed;
    eth_duplex_t duplex;
    (void)ctx;
    if (!s_link_up) return 0;
    if (esp_eth_ioctl(s_eth, ETH_CMD_G_SPEED, &speed) != ESP_OK) return 0;
    if (esp_eth_ioctl(s_eth, ETH_CMD_G_DUPLEX_MODE, &duplex) != ESP_OK) return 0;
    *mbps = (speed == ETH_SPEED_100M) ? 100 : 10;
    *full = (duplex == ETH_DUPLEX_FULL);
    *autoneg = 1;
    return 1;
}
```

7.2.4 Start the Adapter

Procedure: Start the adapter on ESP-IDF

1. Call `eth_start`.
2. Set all fields of `cfg` (chapter 4).
3. Set `cfg.link_info` to your callback.
4. Optional: call `eip_esp_set_netif` to select the interfaces.
5. Call `eip_adapter_start`.

```
#include "eip.h"
#include "eip_esp_idf.h"

eth_start();
cfg.link_info = link_info;       /* with the rest of cfg, from chapter 4 */
eip_esp_set_netif(&s_netif, 1);  /* optional: default is ETH_DEF, WIFI_STA_DEF,
    WIFI_AP_DEF */
eip_adapter_start(&cfg);
```

Call `eip_esp_set_netif` before `eip_adapter_start`. The function copies the list, and the netifs must stay valid while the adapter runs.

7.2.5 Options

Put the options in compile definitions for all components. In the top-level `CMakeLists.txt`, set them after the line that includes `project.cmake` and before `project()`:

```
idf_build_set_property(COMPILE_DEFINITIONS "EIP_ESP_TASK_PRIORITY=12" APPEND)
idf_build_set_property(COMPILE_DEFINITIONS "EIP_ESP_TASK_WDT=1" APPEND)
```

7.2.6 Platform Notes

- Use wired Ethernet for Class 1 in production. Wi-Fi latency peaks are frequently longer than a timeout of 8 × RPI.
- The adapter task waits in `select()`, so it starts as soon as a packet arrives. While idle, it still does a pass at least each `EIP_ADAPTER_IDLE_WAIT_MS` (5 ms).

Table 7.1. ESP-IDF port options

Define	Default	Effect
`EIP_ESP_TASK_STACK`	6144	Adapter task stack in bytes (or `EIP_ADAPTER_TASK_STACK`).
`EIP_ESP_TASK_PRIORITY`	12	Adapter task priority (or `EIP_ADAPTER_TASK_PRIORITY`).
`EIP_ESP_TASK_CORE`	`tskNO_AFFINITY`	Pins the adapter task to a core.
`EIP_ESP_TASK_WDT`	0	1 adds the adapter task to the task watchdog.

- Arduino-ESP32 runs on ESP-IDF and can use the same files. Make a library directory whose `src/` contains `include/*`, `src/*`, `ports/bsd_sockets/*`, and `ports/esp-idf/*`. Do not include the other ports, because Arduino compiles all files that it finds. This setup has not been tested.
- The example `examples/esp_idf_adapter` uses `protocol_examples_common`. For that example only, run `idf.py menuconfig` and set the network under Example Connection Configuration.

7.3 Teknic ClearCore

`examples/clearcore_adapter` compiles and links with the arm-gcc 6.3 of Microchip Studio, against the Enhanced ClearCore Library, a fork of ClearCore library 1.7. The adapter has run on a ClearCore with a Micro850 PLC. The example itself has not been run.

ClearCore runs on bare metal (no RTOS), on the raw API of lwIP with `NO_SYS=1`. QuickEIP uses lwIP directly through `ports/lwip_raw`. It does not use the `EthernetTcp*` and `EthernetUdp` classes. Those classes have a 600-byte TCP buffer and no broadcast send, and EtherNet/IP needs more.

Procedure: Set up a ClearCore project

1. Copy `ProjectTemplate`, `libClearCore`, and `LwIP` from the ClearCore library directory to a location where you can write.
2. Put your QuickEIP directory beside them. The four directories must be at the same level, because the paths below are relative.
3. Open `ProjectTemplate.atsln`.
4. Make sure that the three projects load.
5. Add the files with Add → Existing Item → Add As Link: `src/*.c` (all files), `ports/lwip_raw/eip_hal_lwip_raw.c`, and `ports/clearcore/eip_hal_clearcore.cpp`.
6. Open Project Properties → Toolchain. Do steps 7–10 in the Debug and the Release configurations.
7. In ARM/GNU C Compiler → Directories and ARM/GNU C++ Compiler → Directories, add `../../QuickEIP/include` and `../../QuickEIP/src`.

8. In ARM/GNU C Compiler → Directories only, add `../../LwIP/LwIP/src/include` and `../../LwIP/LwIP/port/include`.
9. In ARM/GNU C Compiler → Symbols and ARM/GNU C++ Compiler → Symbols, add `EIP_ADAPTER_MAX_CLIENTS=2`.
10. Keep the symbols `DEBUG` and `NDEBUG` and the flag `-std=gnu11` that are in the template.
11. Replace `main.cpp` of the template with your main loop.

NOTE: The template sets the lwIP include paths for C++ only. The lwIP port is a C file. Without step 8, the build stops at `lwip/ip_addr.h`.

CAUTION

Add `EIP_ADAPTER_MAX_CLIENTS=2` to the C and the C++ symbols. It changes the size of structures, so the two compilers must agree. The default of 4 is too many for this lwIP build.

Paths are relative to the `Debug` or `Release` output directory. To check a compile without Studio, run `arm-none-eabi-gcc` or `arm-none-eabi-g++` with `-c` from `ProjectTemplate/Debug` and these flags. The toolchain is in `Atmel/Studio/7.0/toolchain/arm/arm-gnu-toolchain/bin`, and *Packs* is `Atmel/Studio/7.0/Packs`:

```
-mthumb -mcpu=cortex-m4 -mfloat-abi=hard -mfpu=fpv4-sp-d16
-D__SAME53N19A__ -DEIP_ADAPTER_MAX_CLIENTS=2
-I../../QuickEIP/include -I../../QuickEIP/src -I../../QuickEIP/ports/clearcore
-I../../libClearCore/inc -I../../LwIP/LwIP/src/include -I../../LwIP/LwIP/port/include
-I<Packs>/atmel/SAME53_DFP/1.1.118/include
-I<Packs>/arm/CMSIS/4.5.0/CMSIS/Include
-I<Packs>/arm/CMSIS/5.4.0/CMSIS/Core/Include
```

7.3.1 The Main Loop

The full example is in `examples/clearcore_adapter/main.cpp`:

```
#include "ClearCore.h"
#include "eip.h"

while (!EthernetMgr.PhyLinkActive()) {                // wait for the cable
    Delay_ms(100);
}
EthernetMgr.Setup();
if (!EthernetMgr.DhcpBegin()) {
    EthernetMgr.LocalIp(IpAddress(192, 168, 1, 50));  // fallback static address
}
if (eip_adapter_start(&cfg) != EIP_OK) {              // poll mode: starts no task
    while (true) { EthernetMgr.Refresh(); }           // stop: outputs stay off
}

while (true) {
    eip_adapter_poll();             // every pass
    EthernetMgr.Refresh();
```

```
    // motion, I/O, eip_adapter_read / eip_adapter_write ...
}
```

7.3.2 Platform Notes

Call `eip_adapter_poll` frequently. T→O packets go out from inside the poll call. Thus the loop time becomes RPI jitter. Keep the loop time much shorter than your RPI.

The lwIP build is small. It has a 4 KB heap (`MEM_SIZE`), five TCP PCBs, and eight listening PCBs. The adapter uses one listening PCB plus one PCB for each explicit session. A scanner session uses one more. This is why you set `EIP_ADAPTER_MAX_CLIENTS` to 2 or 3. For more, compile the LwIP project again with a larger `MEM_SIZE` and `MEMP_NUM_TCP_PCB`. Its `lwipopts.h` uses `#ifndef`, so `-D` operates.

Blocking scanner calls. They call `EthernetMgr.Refresh()` while they wait, and motion continues from interrupts. But your main loop does not run until the call returns. Keep `timeout_ms` short.

Interrupts. Do not call QuickEIP from an interrupt handler. The lwIP callbacks run inside `EthernetMgr.Refresh()` in the main context. This is why the port needs no lock.

Socket table. `EIP_LWIP_MAX_SOCKETS` (12), `EIP_LWIP_UDP_QUEUE` (8), and `EIP_LWIP_ACCEPT_QUEUE` (4) set the size of the raw-API socket table.

Link report. The port reports 100 Mbit/s full duplex while the PHY link is up.

Multicast T→O. The port sets TTL 1 only in lwIP builds with `LWIP_MULTICAST_TX_OPTIONS`. The standard build does not have this option. Thus use a point-to-point connection.

Host name. lwIP on ClearCore has no host name. Call `eip_clearcore_set_host_name` at start-up, and again from `apply_interface` when a scanner sets attribute 6.

> **△ WARNING**
>
> On ClearCore, the main loop also controls motion. A blocking scanner call stops the main loop until it returns. Do not make scanner calls while motion needs service from the main loop. Keep `timeout_ms` short.

The same three groups of files should also operate as an Arduino library with the ClearCore wrapper. This has not been tested.

7.4 Linux, macOS, and WSL

Section 3.2 gives the build procedure. The build makes `libquickeip.a` (the core and the POSIX port), the tests, and `eiptool`. Appendix D describes `eiptool`.

- Set `EIP_NETIF=eth1` to select the interface that the adapter reports and that the scan uses first. This applies to all programs that use the POSIX port. The port ignores an unknown name and uses the default interface.
- Ports 44818 and 2222 are above 1024, so you do not need root access. Another EtherNet/IP program on the same host blocks them.

- Under WSL2, the host is behind NAT. A scan to the LAN operates, but PLCs on your LAN cannot get to a WSL adapter. Use mirrored networking.

7.5 Windows

The port was tested with Visual Studio 2022 on Windows 11, as x64 and Win32. The port is `ports/windows/eip_hal_windows.c`. It needs Windows 7 or later. Section 3.2 gives the build procedure.

Procedure: Add QuickEIP to a Windows project

1. Compile `src/*.c` and `ports/windows/eip_hal_windows.c`.
2. Add `include/` and `src/` to the include path.
3. Link `ws2_32`, `iphlpapi`, and `winmm`.

Table 7.2. Windows port options

Define	Default	Effect
`EIP_WIN_POLL_MODE`	Off	Starts no adapter thread. Your loop calls `eip_adapter_poll`.
`EIP_ADAPTER_TASK_PRIORITY`	0	A `THREAD_PRIORITY_*` value for the adapter thread. 0 selects `THREAD_PRIORITY_HIGHEST`.
`EIP_ADAPTER_TASK_STACK`	0	Adapter thread stack in bytes. 0 is the system default.
`EIP_WIN_MAX_SOCKETS`	64	Sockets open at the same time.
`EIP_WIN_MAX_INTERFACES`	16	Interfaces reported.
`EIP_WIN_NETIF_REFRESH_MS`	2000	Interval at which the port reads the interface list again, outside the adapter thread.

7.5.1 Platform Notes

Select the interface on a PC with more than one. The adapter reports interface 0 as its address, in ListIdentity and in the O→T socket address of a Forward_Open. Interface 0 is the first one that is up and has a default gateway. Frequently, this is not the machine network. Set `EIP_NETIF` to the friendly name or the IPv4 address of the adapter interface, for example `EIP_NETIF="Ethernet 2"` or `EIP_NETIF=10.168.41.1`. The port keeps a named interface also while its cable is disconnected, and reports the link as down. Scanner calls do not need this setting.

Firewall. An adapter needs inbound TCP 44818, UDP 44818, and UDP 2222 for your program. A scanner that runs Class 1 needs inbound UDP on its local port. Use `Get-NetConnectionProfile` to find the profile of the network.

A new executable can lose its first replies. Third-party security software can stop inbound UDP for the first seconds that a new program runs, also with Windows Firewall off. On a test PC with AVG, each new binary failed its first `eiptool identity`, and then answered in approximately 45 ms. If a new build times out, or a scan finds too few devices, run it again before you look for a network fault.

Busy ports. Other EtherNet/IP software on the PC, such as RSLinx, can hold 44818 or 2222. Each bind is exclusive. The adapter then cannot open the ports, and `listening` stays 0.

Timing. The port sets a 1 ms system timer period for the process from its first call. This uses more power on a laptop. On the test PC, a 10 ms RPI had a mean of 10.00 ms and a maximum of 11.83 ms over 1,500 packets, with the two ends on the same host.

Link report. Windows gives the speed. It does not give duplex or autonegotiation, so the port reports full duplex and autonegotiated while the link is up.

Not tested. Multicast T→O, and a real PLC connected to a Windows adapter.

△ WARNING

Windows is not a real-time operating system. A stall of approximately 100 ms was measured on a bench PC. Set `timeout_floor_ms` to 500 or more. Do not use a Windows adapter for a safety function.

CHAPTER 8

Porting to a New Platform

A port is one source file, sometimes two, that implements the functions in `include/eip_hal.h`. The core calls these functions directly. There is no function table. This chapter tells you what each function must do and how to check a new port.

8.1 Select a Starting Point

Start from the port that is nearest to your platform. Table 8.1 shows which port to use.

Table 8.1. Starting points for a new port

Your platform has	Start from	You write
BSD sockets (`socket`, `select`, `fcntl`): lwIP with `LWIP_SOCKET`, Zephyr, Linux, FreeRTOS+TCP with its BSD layer	`ports/bsd_sockets/eip_hal_bsd_sockets.c`, without changes	A platform file like `ports/esp-idf/eip_hal_esp_idf.c`: clock, lock, interfaces, and task
The lwIP raw API, `NO_SYS=1`, no socket layer: STM32Cube bare metal, SAM, other boards like ClearCore	`ports/lwip_raw/eip_hal_lwip_raw.c`, without changes	A platform file like `ports/clearcore/eip_hal_clearcore.cpp`: clock, yield (your stack poll), and interfaces
A vendor socket API that is neither (NetBurner)	`ports/netburner/eip_hal_netburner.cpp`	The full contract
Winsock	`ports/windows/eip_hal_windows.c`, without changes	Nothing

The platform file is short. For ESP-IDF it is approximately 190 lines with comments, and for ClearCore approximately 130.

8.2 General Rules of the Contract

Handles. A socket handle is an `int`. A negative value is not valid, and `EIP_SOCK_INVALID` is -1. The core records which handles it owns, so 0 is a valid handle.

Addresses. An IPv4 address is `uint8_t[4]` in network order, for example `{192,168,1,10}`. A port number is a plain integer in host order.

Timeouts. `timeout_ms == 0` means "do not wait." Return what is ready now.

Blocking waits. Each blocking wait must keep the network stack in operation. On bare metal, call the stack poll inside the loop. This is the purpose of `eip_hal_yield`.

Return codes. `EIP_OK` (0) or a negative `EIP_ERR_*`, unless the function gives a different rule.

Table 8.2. Port functions: time, scheduling, and lock

Function	Must
`eip_hal_millis`	Return a free-running millisecond counter that wraps at 2^{32}. The core uses only differences. T→O packets are timed from it, so use a real 1 ms source if the platform has one.
`eip_hal_clock_resolution_ms`	Return the real step of the counter in milliseconds (1 for a 1 kHz source). The adapter never sets a Class 1 timeout shorter than two steps.
`eip_hal_yield`	Let other work run for a short time. On an RTOS, sleep one tick. On bare metal, run the poll of the network stack, for example `EthernetMgr.Refresh()`, or `sys_check_timeouts()` plus `ethernetif_input()`.
`eip_hal_lock`, `eip_hal_unlock`	Give mutual exclusion between the adapter context and application tasks. The lock is not nested, is held for microseconds, and is never held during a socket call. The core can read `eip_hal_millis` while it holds the lock. Single-threaded ports leave the two functions empty.

8.3 Time and Scheduling

CAUTION

Use a clock with a real 1 ms step if the platform has one. On Windows, `GetTickCount64` steps approximately each 16 ms. With that clock, the adapter sent T→O at approximately half the rate, and the connection timed out. `tests/test_hal.c` measures the step and fails a clock that is coarser than it reports.

8.4 Network Interfaces

`eip_hal_netif_get(index, info)` fills in the data of interface `index`. Index 0 is the primary interface. These rules apply:

- Return `EIP_ERR_ARG` after the last interface.
- An interface without an address still returns `EIP_OK`, with `ip` = 0.0.0.0. The POSIX port is an exception. It lists only interfaces that are up and have an IPv4 address.
- Set to zero all fields that you do not know.
- Set `link_known = 1` when you can report `link_up`. If you do not, the server uses "has an address" as "link up, 100 Mbit/s full duplex." Fill `speed_mbps`, `full_duplex`, and `autonegotiated` with real values if you can.

The core uses the result in three ways:

1. The adapter calls the function each 100 ms for `netif_index`. It fills in its Identity, TCP/IP Interface, and Ethernet Link objects.
2. The scan reads indexes 0, 1, 2, and so on, to find broadcast addresses.
3. The MAC address sets the default serial number.

8.5 TCP

Table 8.3. Port functions: TCP

Function	Must
`eip_hal_tcp_connect`	Connect, and wait up to `timeout_ms`. Return a handle or `EIP_SOCK_INVALID`.
`eip_hal_tcp_listen`	Listen on all addresses. Return a handle or `EIP_SOCK_INVALID`.
`eip_hal_tcp_accept`	**Never block.** Return a connected handle and fill `peer_ip`, or return `EIP_SOCK_INVALID` when no client waits.
`eip_hal_tcp_send`	Queue **all** `len` bytes. Return `EIP_ERR_IO` if the connection is broken, or if the bytes cannot be queued in a few hundred milliseconds. Do not block for an unlimited time.
`eip_hal_tcp_recv`	Return the number of bytes read (> 0), 0 if nothing arrived in time, or `EIP_ERR_IO` after the peer closed or the connection failed. With `timeout_ms == 0`, return immediately.
`eip_hal_tcp_local_ip`	Return the local address of the connection. It goes into the T→O socket address of a Forward_Open.
`eip_hal_tcp_close`	Close the socket and release the handle. This applies to connected and listening sockets.

For adapter sessions, set TCP_NODELAY and keep-alive (approximately 30 s idle, three probes). Table 8.4 shows what the shipped ports do.

Table 8.4. TCP options in the shipped socket layers

Layer	TCP_NODELAY	Keep-alive
`bsd_sockets`	All connections	Accepted sockets only. The 30 s setting needs `TCP_KEEPIDLE`, `TCP_KEEPINTVL`, and `TCP_KEEPCNT`.
`lwip_raw`	All connections	Only in lwIP builds with `LWIP_TCP_KEEPALIVE`.
`netburner`	No	No. The slot of a lost peer comes back through eviction.

8.6 UDP

CAUTION

Do not use `SO_REUSEADDR`. The scanner finds a free port in UDP 2223–2231 because a bind to a port in use fails. The same failure stops an explicit `local_port` of 2222 while an adapter runs on the same device.

A socket bound to the any-address must receive broadcasts. The adapter answers broadcast ListIdentity requests on UDP 44818.

8.7 Wait and the Adapter Runner

The server itself is static. `eip_server_service` needs approximately 1 KB of stack, plus the call depth of the socket layer of your platform.

Table 8.5. Port functions: UDP

Function	Must
`eip_hal_udp_open`	Bind on all addresses. `local_port == 0` means any free port. **Fail if the port is already in use.** `EIP_UDP_BROADCAST` permits broadcast sends. `EIP_UDP_MULTICAST_TX` permits multicast sends with TTL 1 on the primary interface, where the stack can set them.
`eip_hal_udp_sendto`	Send one datagram. Return `EIP_OK` or `EIP_ERR_IO`.
`eip_hal_udp_recvfrom`	Receive one datagram. Return its length, 0 if nothing arrived in time, or `EIP_ERR_IO`. Cut a long datagram to `cap`. `ip` and `port` can be null.
`eip_hal_udp_close`	Close the socket and release the handle.

Table 8.6. Port functions: wait and runner

Function	Must
`eip_hal_wait`	Wait until one of the handles can be readable (or a listener has a connection), or until the timeout. Skip handles that are not valid. The function can return early. Without `select()`, a loop of `eip_hal_yield` and readiness checks is correct.
`eip_hal_adapter_start_task`	Called one time by `eip_adapter_start`. **RTOS:** make a task that runs `for (;;) eip_server_service(server, EIP_ADAPTER_IDLE_WAIT_MS);`. Give it a stack of 4–6 KB. **Bare metal:** do nothing and return `EIP_OK`. The application then calls `eip_adapter_poll`.

8.8 Check a New Port

Do these checks before you connect a PLC.

Procedure: Check a new port

1. Compile with warnings on (`-Wall -Wextra`). The core compiles without warnings on the compilers of the shipped ports.
2. If your platform can run it, run `tests/test_hal.c`. Make sure that all checks pass.
3. On the target, start the test adapter with `eip_adapter_start(0)`.
4. On a PC on the same network, build the POSIX version of `eiptool` (section 3.2).
5. Run the commands in the listing below. Make sure that each result is correct.
6. Test Class 1 from the PC. Use `eip_io_open` with config 150, output 100, input 101, and 8 bytes in each direction, as in `tests/test_loopback.c`.
7. Let the connection run for some minutes. On the target, monitor `rx_packets`, `timeouts`, and the `rx_dropped_*` counters in the snapshot.
8. On the PC, run `eiptool adapter`.
9. From the target, call `eip_list_identity`, `eip_session_open`, `eip_get_attribute`, and `eip_io_open` against the PC.
10. Test with a real PLC and an EDS file that agrees with your assemblies.

```
./build/eiptool scan                      # finds "EIP Test Adapter"
./build/eiptool identity <target-ip>
./build/eiptool get <target-ip> 1 1 7     # Identity product name
./build/eiptool get <target-ip> 4 101 3   # produced data: 11 22 33 44 55 66 77 88
./build/eiptool get <target-ip> 0xF5 1 5 # TCP/IP interface configuration
```

If a check fails, compare your port with the POSIX port. It is the reference, and `test_loopback` runs all paths through it.

8.9 Rules for Changes to the Core

If you change the core itself, obey these rules:

- Keep `src/` free of OS calls. All OS access goes through `eip_hal.h`.
- Add a test with each change of behavior: `tests/test_core.c` for the codec and the adapter core, `tests/test_session.c` for scanner sessions, and `tests/test_hal.c` for the port contract.
- Use `/* */` comments in ASCII. Tell why, not what.

CHAPTER 9

Troubleshooting

This chapter helps you find the cause of a fault. Start with the snapshot of the adapter, or with the CIP status of the scanner. They usually show the cause.

9.1 Fault Isolation Procedure

Procedure: Isolate a Class 1 fault

1. Call `eip_adapter_snapshot`.
2. If `listening` is 0, go to table 9.1, first row.
3. If the PLC shows a module fault, read its status code.
4. Find the code in table 4.3.
5. If `opens` increases and `timeouts` increases with it, examine the `rx_dropped_*` counters.
6. If `rx_dropped_source` increases, examine the addresses (section 6.5).
7. If `rx_dropped_seq` increases, look for duplicate or old packets on the network.
8. If the consumed data is zero but the PLC sends data, examine `run`. The PLC is possibly in Program mode.

9.2 Symptoms and Causes

Table 9.1. Symptoms and their probable causes

Symptom	Probable cause and remedy
`listening` stays 0	Another program holds TCP or UDP 44818 or UDP 2222. Or the interface has no address yet. The server tries again each second.
The PLC shows a module fault with code `16#0127` or `16#0128`	The EDS connection sizes do not agree with the assembly sizes. See section 4.4. Remember the 4-byte run/idle header on O→T.
Class 1 opens, then stops after less than a second, and `rx_dropped_source` increases	Use the IP address that the adapter reports, not `127.0.0.1` or another address of the host. See section 6.5.
`eip_io_open` returns `EIP_ERR_IO` after a lost connection	Close and open the session also. See section 6.5.1.

Symptom	Probable cause and remedy
ESP-IDF: `Failed to resolve component ... unknown name`	The key in `idf_component.yml` is not the same as the QuickEIP directory name. See section 7.2.
ClearCore: `lwip/ip_addr.h: No such file or directory`	The C compiler does not have the LwIP paths. See section 7.3, step 8.
The connection times out each few seconds	O→T data does not arrive, or the adapter refuses it. `rx_dropped_source` counts packets from a wrong address (NAT, hosts with more than one address). `rx_dropped_seq` counts duplicate or old sequence numbers. On Wi-Fi, examine the latency.
`opens` increases and `timeouts` increases with it	The same as the previous row. Also make sure that no firewall stops UDP 2222 or TCP and UDP 44818.
Outputs read zero, but the PLC sends data	The PLC is in Program mode (idle). This is the design.
Scanner `EIP_ERR_CIP` with `0x05` or `0x04`	The tag or the path is not found (`0x05`), or it is malformed (`0x04`). Examine the tag name and the slot (−1 or a backplane slot).
Scanner `EIP_ERR_CIP` with `0x16`	The object does not exist, for example a wrong assembly instance.
Scanner `EIP_ERR_CIP` with `0x01` / `0x0204` on a routed session	The router did not get to the slot in time. Examine the slot number, and make sure that a module is in it.
Scanner calls return `EIP_ERR_STATE` after an error	The session closed itself after a broken packet. Open it again.
A scan finds nothing	Broadcasts do not cross routers or VLANs. Use `eip_list_identity` with an address. On Windows, also run a new program a second time (section 7.5).
A new Windows build times out on its first call	Security software can stop the first replies of a new executable. Run the program again.

9.3 Common Mistakes

Table 9.2 lists the mistakes that cost the most time.

Table 9.2. Common mistakes

Area	Mistake and correction
Slot	A zeroed `EipOpenConfig` has slot 0, which routes across a backplane. Set `slot = -1` for a device that you reach directly.
Sizes	Enter assembly bytes in an EDS file or a PLC. The 2-byte sequence count and the 4-byte run/idle header are added on the wire.
Direction	Consumed is what the PLC writes. Produced is what the PLC reads. The test set uses 100 as consumed, the opposite of the examples.
Class 1 address	Class 1 needs the address that the adapter reports. It fails on `127.0.0.1` and on other addresses of the same host.
Lost connection	Recovery needs a new session also. `eip_session_is_open` does not find a lost peer.
Configuration	The settings in `eip_config.h` change structure sizes. All C and C++ files must see the same values.
Start order	Set `link_info` and the other configuration fields before `eip_adapter_start`, which copies them.
Poll	Only poll-mode ports call `eip_adapter_poll`. Do not call it on an RTOS port.

Appendix A

API Reference

This appendix describes each public function. The source is the comments in `include/`. Unless an entry gives a different rule, a function that returns `int` returns `EIP_OK` or a negative `EIP_ERR_*` code (table C.1). In a parameter pair such as `data` and `data_len`, `data_len` is the capacity on input and the byte count on output.

A.1 Utilities (`eip.h`)

eip_status_text

```
const char *eip_status_text(int code);
```

Gives a short English text for an `EIP_OK` or `EIP_ERR_*` code.

Returns: A static string. Never null.

eip_parse_ipv4

```
int eip_parse_ipv4(const char *text, uint8_t ip[4]);
```

Parses dotted-quad IPv4 text, for example `"192.168.1.10"`, into four bytes in network order.

Returns: `EIP_OK` or `EIP_ERR_ARG`.

eip_store_le16

```
static inline void eip_store_le16(uint8_t *p, uint16_t value);
```

Writes `value` at `p` in little-endian order (two bytes).

eip_store_le32

```
static inline void eip_store_le32(uint8_t *p, uint32_t value);
```

Writes `value` at `p` in little-endian order (four bytes).

eip_load_le16

```
static inline uint16_t eip_load_le16(const uint8_t *p);
```

Reads a little-endian 16-bit value at `p`.

eip_load_le32

```
static inline uint32_t eip_load_le32(const uint8_t *p);
```

Reads a little-endian 32-bit value at `p`. To get a signed DINT, cast the result to `int32_t`.

A.2 Scanner Sessions (`eip.h`)

eip_session_open

```
int eip_session_open(EipSession *session, const EipOpenConfig *config);
```

Connects to the device and registers a session. The function writes over `*session`, so close an open session first. `config->slot` must be −1 to 255. The call blocks for up to `timeout_ms` for the connection, and again for the reply.

Returns: `EIP_OK`. `EIP_ERR_CONNECT` if TCP cannot connect. `EIP_ERR_PROTOCOL` if the device refuses the registration. Then `eip_last_encap_status` gives the reason.

eip_session_close

```
void eip_session_close(EipSession *session);
```

Sends UnregisterSession without a wait for an answer, and closes the socket. Safe on a closed or zeroed session. Close all `EipIo` connections on the session first, because `eip_io_close` needs the session to send Forward_Close.

eip_session_set_slot

```
void eip_session_set_slot(EipSession *session, int16_t slot);
```

Changes the routing of later requests on an open session. The function ignores a slot outside −1 to 255.

eip_session_is_open

```
int eip_session_is_open(const EipSession *session);
```

Returns: 1 if the session is registered here, else 0. Safe on a zeroed structure. It does not find a lost peer. After `EIP_ERR_IO`, close the session and open it again.

eip_last_cip_status

```
uint8_t eip_last_cip_status(const EipSession *session);
```

Returns: The general status of the last CIP reply. 0 for a null session.

eip_last_cip_extended

```
uint16_t eip_last_cip_extended(const EipSession *session);
```

Returns: The extended status of the last CIP reply. 0 for a null session.

eip_last_encap_status

```
uint32_t eip_last_encap_status(const EipSession *session);
```

Returns: The status field of the last encapsulation reply. 0 for a null session.

A.3 Discovery (`eip.h`)

eip_list_identity

```
int eip_list_identity(const char *ip, EipIdentity *identity, uint32_t timeout_ms);
```

Sends a UDP ListIdentity to one address. It does not need an open session. A `timeout_ms` of 0 waits 2 s.

Returns: `EIP_OK`, or `EIP_ERR_TIMEOUT` if no reply arrives.

eip_scan_devices

```
int eip_scan_devices(EipDevice *devices, uint16_t *devices_len, uint32_t timeout_ms);
```

Sends a broadcast ListIdentity on each interface and to the global broadcast address. `devices_len` is the array capacity on input and the number of different devices on output. A `timeout_ms` of 0 waits approximately 3 s, sends again each second, and returns 0.5 s after the last reply. A device is marked `micro800` if its vendor is 1 and its name contains `"2080-"`, `"2085-"`, or `"micro8"`.

Returns: `EIP_OK`. A count of 0 means that the scan ran and no device answered.

eip_device_kind

```
int eip_device_kind(uint16_t vendor_id, const char *product_name);
```

Sorts a ListIdentity product name with Rockwell catalog heuristics. The function tests for Micro800 before CompactLogix.

Returns: `EIP_KIND_MICRO800`, `EIP_KIND_COMPACTLOGIX`, or `EIP_KIND_ASSEMBLY`.

eip_list_symbols

```
int eip_list_symbols(EipSession *session, EipSymbol *symbols, uint16_t *symbols_len);
```

Reads the Symbol object (class `0x6B`, service `0x55`) on an open session. `symbols_len` is the capacity on input and the count stored on output. Open a Micro800 with slot −1. If a later page fails, the count still gives the symbols stored before the failure.

A.4 Explicit Messages and Tags (`eip.h`)

eip_cip

```
int eip_cip(EipSession *session, uint8_t service, const uint8_t *path, uint16_t
path_len, const uint8_t *request, uint16_t request_len, uint8_t *response, uint16_t
*response_len, uint8_t *cip_status, uint16_t *extended_status);
```

Sends one explicit message and waits for the reply. `path` is the raw path bytes, without the path-size word. `response` and `response_len` can both be 0 to discard the reply data. `cip_status` and `extended_status` can be 0. The encoded request has a limit of 480 bytes.

Returns: `EIP_OK`. `EIP_ERR_CIP` for a general status that is not 0, except `0x06` (partial transfer), which gives `EIP_OK` with the status in `*cip_status`. `EIP_ERR_TOO_BIG` if the reply data does not fit in `response`.

eip_get_attribute

```
int eip_get_attribute(EipSession *session, uint16_t class_code, uint16_t instance,
uint16_t attribute, uint8_t *data, uint16_t *data_len, uint8_t *cip_status);
```

Sends Get_Attribute_Single.

eip_set_attribute

```
int eip_set_attribute(EipSession *session, uint16_t class_code, uint16_t instance,
uint16_t attribute, const uint8_t *data, uint16_t data_len, uint8_t *cip_status);
```

Sends Set_Attribute_Single. `data` is the attribute value, without a type prefix.

eip_tag_read

```
int eip_tag_read(EipSession *session, const char *tag, uint16_t elements, uint8_t *data,
uint16_t *data_len, uint16_t *cip_type, uint8_t *cip_status);
```

Reads `elements` elements of a Logix tag. Tag names use a period for members and `[n]` or `[n,m]` for elements. `cip_type` receives the CIP type of the tag. `cip_type` and `cip_status` can be 0. A reply too large for one packet continues with Read Tag Fragmented until the device reports the end.

eip_tag_write

```
int eip_tag_write(EipSession *session, const char *tag, uint16_t cip_type, uint16_t
elements, const uint8_t *data, uint16_t data_len, uint8_t *cip_status);
```

Writes `elements` elements of type `cip_type`. `data` is little-endian element data. `cip_status` can be 0. Data larger than one request (no more than 400 bytes, less for a long tag path) goes out with Write Tag Fragmented. An error during the write can leave the earlier fragments written.

A.5 Class 1 Originator (`eip.h`)

eip_io_open

```
int eip_io_open(EipSession *session, EipIo *io, const EipIoConfig *config);
```

Sends a Forward_Open and binds a UDP socket. The connection size sent to the target is 2 bytes of sequence count, plus 4 bytes if `run_idle` is set, plus the assembly size. The function binds UDP 2223–2231 unless `local_port` is set. Assemblies have a limit of 600 bytes in each direction.

Returns: `EIP_OK`. `EIP_ERR_CIP` if the target refuses the Forward_Open. `EIP_ERR_TOO_BIG` if a size is more than the limit. `EIP_ERR_STATE` if the session is not open. `EIP_ERR_IO` if no UDP port can be bound, or if the session is broken. After a lost connection, close and open the session (section 6.5.1).

eip_io_produce

```
int eip_io_produce(EipIo *io, const uint8_t *data, uint16_t data_len, int run);
```

Sends one O→T packet. Call it one time each RPI. The library does not set the interval. `data_len` must be equal to `output_bytes`. `run` sets the run/idle header, if the connection has one.

eip_io_consume

```
int eip_io_consume(EipIo *io, uint8_t *data, uint16_t *data_len, uint32_t timeout_ms);
```

Waits for the next T→O packet of this connection and copies `input_bytes` of it. The function skips packets for other connections. A `timeout_ms` of 0 waits 1 s. There is no non-blocking form.
Returns: `EIP_ERR_TIMEOUT` if no packet arrives. This is how a lost connection shows, because `eip_io_produce` continues to succeed.

eip_io_close

```
int eip_io_close(EipSession *session, EipIo *io);
```

Sends Forward_Close if the session is still open. Then it releases the UDP socket, whatever the result.

A.6 Built-In Adapter (eip.h)

eip_adapter_start

```
int eip_adapter_start(const EipAdapterConfig *config);
```

Copies what it needs from `config` and listens on TCP and UDP 44818 and UDP 2222. `config` can be 0 for the test identity and assemblies. RTOS ports start a task. Poll-mode ports start nothing.
Returns: `EIP_OK`. `EIP_ERR_STATE` if the adapter is already started. `EIP_OK` does not mean "listening": see `EipAdapterView.listening`.

eip_adapter_poll

```
uint32_t eip_adapter_poll(void);
```

Does one non-blocking pass of the adapter. Call it from the main loop on poll-mode ports only.
Returns: The milliseconds until the adapter next needs a call.

eip_adapter_snapshot

```
int eip_adapter_snapshot(EipAdapterView *view);
```

Copies the adapter state into `*view`. Safe from any task.
Returns: `EIP_OK`, or `EIP_ERR_STATE` before a successful start.

eip_adapter_read

```
int eip_adapter_read(uint16_t instance, uint8_t *data, uint16_t *data_len);
```

Copies the data of an assembly. A capacity smaller than the assembly size is `EIP_ERR_TOO_BIG`. A consumed assembly reads as zeros after a timeout or a close and while the scanner is idle. Before the first connection, it reads as its initial data. Safe from any task.
Returns: `EIP_OK`, `EIP_ERR_PATH` for an unknown instance, `EIP_ERR_TOO_BIG`, or `EIP_ERR_STATE` before a start.

eip_adapter_write

```
int eip_adapter_write(uint16_t instance, const uint8_t *data, uint16_t data_len);
```

Replaces the data of an assembly, usually a produced one. The next T→O packet carries it. `data_len` must be equal to the assembly size. Safe from any task.

A.7 Adapter Server (`eip_server.h`)

An `EipServer` is approximately `EIP_MAX_PACKET` × (`EIP_ADAPTER_MAX_CLIENTS` + 3) bytes. `init`, `poll`, `service`, and `close` run in one context. `read`, `write`, and `snapshot` are safe from any task.

eip_server_init

```
int eip_server_init(EipServer *server, const EipAdapterConfig *config);
```

Sets up the adapter and tries to open its sockets. `config` can be 0 for the test identity and assemblies. The poll function tries again each second to open sockets that failed. Thus `EIP_OK` does not mean "listening."

eip_server_poll

```
uint32_t eip_server_poll(EipServer *server);
```

Does one non-blocking pass. It opens the sockets if necessary, reads the interface values again, takes Class 1 input, answers UDP 44818, serves and accepts TCP clients, checks timeouts, sends the T→O packets that are due, and runs the deferred callbacks.
Returns: The milliseconds until the next T→O packet, with a limit of `EIP_ADAPTER_IDLE_WAIT_MS`.

eip_server_service

```
void eip_server_service(EipServer *server, uint32_t max_wait_ms);
```

Waits for socket activity through `eip_hal_wait`, for the shorter of `max_wait_ms` and the next T→O deadline. Then it polls. This is the body of an adapter task.

eip_server_close

```
void eip_server_close(EipServer *server);
```

Closes all sockets. You can initialize the server again after this call. The call does nothing on a server that is not initialized.

eip_server_snapshot

```
int eip_server_snapshot(EipServer *server, EipAdapterView *view);
```

The same as `eip_adapter_snapshot`, for this instance.

eip_server_read

```
int eip_server_read(EipServer *server, uint16_t instance, uint8_t *data, uint16_t
*data_len);
```

The same as `eip_adapter_read`, for this instance.

eip_server_write

```
int eip_server_write(EipServer *server, uint16_t instance, const uint8_t *data,
uint16_t data_len);
```

The same as `eip_adapter_write`, for this instance.

eip_adapter_default_server

```
EipServer *eip_adapter_default_server(void);
```

Returns: The instance behind `eip_adapter_start` and `eip_adapter_poll`.

A.8 Port Extras

eip_esp_set_netif

```
void eip_esp_set_netif(esp_netif_t *const *netifs, uint8_t count);
```

ESP-IDF only (`ports/esp-idf/eip_esp_idf.h`). Selects the esp_netif handles that the library sees, in order. Index 0 is the one that the adapter reports. Call it before `eip_adapter_start`. Without it, the port uses `ETH_DEF`, `WIFI_STA_DEF`, and `WIFI_AP_DEF`, if they exist. The function copies up to four handles and ignores the others. The netifs must stay valid while the adapter runs. A count of 0 sets the default lookup again.

eip_clearcore_set_host_name

```
void eip_clearcore_set_host_name(const char *name);
```

ClearCore only (`ports/clearcore/eip_clearcore.h`). Sets the host name in TCP/IP Interface attribute 6, up to 64 characters. The function copies the string and cuts a longer name to 64 characters. Call it at start-up with the stored name. Call it again from `apply_interface` when a scanner sets attribute 6. The adapter reads this name again at each interface refresh, so a value that you do not store here goes back to the old value.

A.9 Port Contract (`eip_hal.h`)

A port implements these 19 functions. Chapter 8 gives the full rules.

eip_hal_millis

```
uint32_t eip_hal_millis(void);
```

Free-running millisecond counter. Wraps at 2^{32}.

eip_hal_clock_resolution_ms

```
uint32_t eip_hal_clock_resolution_ms(void);
```

The step of `eip_hal_millis` in milliseconds: 1 on most ports, 50 on a 20 Hz NetBurner tick.

eip_hal_yield

```
void eip_hal_yield(void);
```

Lets other work run: one tick of sleep on an RTOS, the network stack poll on bare metal.

eip_hal_lock

```
void eip_hal_lock(void);
```

Takes the adapter lock. Not nested. Empty on single-threaded ports.

eip_hal_unlock

```
void eip_hal_unlock(void);
```

Releases the adapter lock.

eip_hal_netif_get

```
int eip_hal_netif_get(uint8_t index, EipNetifInfo *info);
```

Fills in the data of interface `index`. `EIP_ERR_ARG` if there is no such interface.

eip_hal_tcp_connect

```
eip_sock_t eip_hal_tcp_connect(const uint8_t ip[4], uint16_t port, uint32_t
timeout_ms);
```

Connects. `EIP_SOCK_INVALID` on failure.

eip_hal_tcp_listen

```
eip_sock_t eip_hal_tcp_listen(uint16_t port, uint8_t backlog);
```

Listens on all interfaces. `EIP_SOCK_INVALID` on failure.

eip_hal_tcp_accept

```
eip_sock_t eip_hal_tcp_accept(eip_sock_t listener, uint8_t peer_ip[4]);
```

Never blocks. A connected socket, or `EIP_SOCK_INVALID`.

eip_hal_tcp_send

```
int eip_hal_tcp_send(eip_sock_t sock, const uint8_t *data, uint16_t len);
```

Queues all `len` bytes, or returns `EIP_ERR_IO`.

eip_hal_tcp_recv

```
int eip_hal_tcp_recv(eip_sock_t sock, uint8_t *buf, uint16_t cap, uint32_t timeout_ms);
```

Byte count (> 0), 0 on timeout, or `EIP_ERR_IO`.

eip_hal_tcp_local_ip

```
int eip_hal_tcp_local_ip(eip_sock_t sock, uint8_t ip[4]);
```

The local address of the connection.

eip_hal_tcp_close

```
void eip_hal_tcp_close(eip_sock_t sock);
```

Closes a connected or listening socket.

eip_hal_udp_open

```
eip_sock_t eip_hal_udp_open(uint16_t local_port, uint8_t flags);
```

Binds a UDP socket. Fails if the port is in use. Flags: `EIP_UDP_BROADCAST`, `EIP_UDP_MULTICAST_TX`.

eip_hal_udp_sendto

```
int eip_hal_udp_sendto(eip_sock_t sock, const uint8_t ip[4], uint16_t port, const
uint8_t *data, uint16_t len);
```

Sends one datagram.

eip_hal_udp_recvfrom

```
int eip_hal_udp_recvfrom(eip_sock_t sock, uint8_t *buf, uint16_t cap, uint8_t ip[4],
uint16_t *port, uint32_t timeout_ms);
```

Length (> 0), 0 on timeout, or `EIP_ERR_IO`. Truncates to `cap`.

eip_hal_udp_close

```
void eip_hal_udp_close(eip_sock_t sock);
```

Closes a UDP socket.

eip_hal_wait

```
int eip_hal_wait(const eip_sock_t *socks, uint8_t count, uint32_t timeout_ms);
```

Waits until a socket can be ready, or until the timeout. > 0 or 0.

eip_hal_adapter_start_task

```
int eip_hal_adapter_start_task(struct EipServer *server);
```

Starts the adapter task (RTOS), or does nothing (poll mode).

A.10 Sans-I/O Core (`eip_adapter.h`)

Use these functions only for an event loop of your own. The core keeps one shared reply buffer, so serialize all calls on all `EipAdapter` objects. Table A.1 summarizes them.

Table A.1. Sans-I/O core functions

Function	Purpose
`eip_adapter_init`	Initialize the core with an IP address and a serial number.
`eip_adapter_set_ip`	Change the reported IP address.
`eip_adapter_set_clock`	Set the ticks per second of the tick count that you supply.
`eip_adapter_set_identity`	Set the Identity object values. The name is cut to 31 characters.
`eip_adapter_clear_assemblies`	Remove the test assemblies. Call it before a connection opens.
`eip_adapter_add_assembly`	Add one assembly from the data pool.
`eip_adapter_set_implicit_only`	Hide an assembly from explicit messages.
`eip_adapter_set_netmask`	Set the mask for the multicast address.
`eip_adapter_set_timeout_floor_ms`	Set the minimum inactivity timeout. 0 removes the floor.
`eip_adapter_set_interface`	Set the TCP/IP Interface values: MAC, gateway, configuration control, and host name.
`eip_adapter_set_callbacks`	Enable settable attributes 3, 5, and 6 and the Identity Reset service.
`eip_adapter_set_dns`	Set the DNS servers and domain in attribute 5.
`eip_adapter_set_link`	Set the Ethernet Link object values.
`eip_adapter_multicast_address`	Calculate the multicast group address.
`eip_adapter_on_encap`	Process one encapsulation packet and build the reply.
`eip_adapter_take_input_from`	Process one Class 1 O→T packet, with a check of the source address.
`eip_adapter_take_input`	The same, without the source check.
`eip_adapter_io_due`	Tell if a T→O packet is due.
`eip_adapter_build_output`	Build the next T→O packet and its destination.
`eip_adapter_check_timeout`	Run the inactivity watchdog.
`eip_adapter_copy`	Copy the data of an assembly.
`eip_adapter_store`	Replace the data of an assembly.

Appendix B

Data Structures

This appendix lists the fields of the public structures in `eip.h`. Treat the fields of `EipSession` and `EipIo` as private. Use the functions to get their values.

B.1 Scanner Structures

Table B.1. `EipOpenConfig`: session parameters

Field	Type	Meaning
`ip`	`const char *`	Dotted IPv4 text, for example `"192.168.1.10"`.
`tcp_port`	`uint16_t`	0 selects 44818.
`slot`	`int16_t`	−1 direct. 0–255: the backplane slot. A zeroed structure has slot 0.
`vendor_id`	`uint16_t`	Your ODVA vendor ID. Used by a Class 1 Forward_Open.
`serial_number`	`uint32_t`	0 is changed to 1.
`timeout_ms`	`uint32_t`	0 selects 5000.

Table B.2. `EipIoConfig`: Class 1 originator parameters

Field	Type	Meaning
`config_instance`	`uint16_t`	Config assembly instance, from the EDS file. 0 is sent as instance 0.
`output_instance`	`uint16_t`	O→T connection point. The target consumes it.
`input_instance`	`uint16_t`	T→O connection point. The target produces it.
`output_bytes`	`uint16_t`	Assembly bytes that you produce. 0 is a heartbeat.
`input_bytes`	`uint16_t`	Assembly bytes that you consume.
`o2t_rpi_us`	`uint32_t`	O→T RPI in microseconds. 0 selects 20,000 (20 ms).
`t2o_rpi_us`	`uint32_t`	T→O RPI in microseconds.
`local_port`	`uint16_t`	0 uses UDP 2223–2231. Port 2222 is for the adapter.
`run_idle`	`uint8_t`	1 adds a 4-byte run/idle header to O→T.
`timeout_multiplier`	`uint8_t`	CIP multiplier 0–7: ×4, ×8, ×16, and so on.

Table B.3. `EipIdentity`: a ListIdentity reply

Field	Type	Meaning
`vendor_id`	`uint16_t`	Vendor ID.
`device_type`	`uint16_t`	Device type.
`product_code`	`uint16_t`	Product code.
`revision_major`, `revision_minor`	`uint8_t`	Revision.
`status`	`uint16_t`	Identity object status word.
`serial_number`	`uint32_t`	Serial number.
`name`	`char[64]`	Product name.
`state`	`uint8_t`	Identity object state.

Table B.4. `EipDevice`: a device found by a scan

Field	Type	Meaning
`ip`	`uint8_t[4]`	Address of the device.
`vendor_id`, `device_type`, `product_code`	`uint16_t`	Identity values.
`revision_major`, `revision_minor`	`uint8_t`	Revision.
`serial_number`	`uint32_t`	Serial number.
`name`	`char[64]`	Product name.
`micro800`	`uint8_t`	1 if the name is a Rockwell Micro800 catalog.

Table B.5. `EipSymbol`: one entry of the Symbol object

Field	Type	Meaning
`name`	`char[96]`	Tag name.
`symbol_type`	`uint16_t`	Symbol type word.
`element_size`	`uint16_t`	Bytes in one element.
`dim0`, `dim1`, `dim2`	`uint32_t`	Array dimensions.
`element_count`	`uint32_t`	Product of the dimensions. A dimension of 0 counts as 1.
`is_array`	`uint8_t`	1 for an array.

B.2 Adapter Structures

Table B.6. `EipAssemblyConfig`: one assembly

Field	Type	Meaning
`instance`	`uint16_t`	Instance number.
`direction`	`uint8_t`	`EIP_ASM_CONSUMED`, `EIP_ASM_PRODUCED`, or `EIP_ASM_CONFIG`.
`size`	`uint16_t`	Bytes. 0 is permitted for a config point.
`name`	`const char *`	Up to 15 characters. Can be 0.
`initial`	`const uint8_t *`	Initial data, or 0 for zeros.
`implicit_only`	`uint8_t`	1: Class 1 only. Explicit Get and Set are refused.

Table B.7. `EipAdapterConfig`: adapter configuration

Field	Type	Meaning
`vendor_id`	`uint16_t`	0 keeps the test identity (vendor 65000, "EIP Test Adapter").
`device_type`	`uint16_t`	12 communications adapter, 43 generic device.
`product_code`	`uint16_t`	Product code.
`revision_major`, `revision_minor`	`uint8_t`	Revision. Use a minor revision of 1 or more for an EDS file.
`product_name`	`const char *`	Up to 31 characters.
`serial_number`	`uint32_t`	0 derives it from the interface MAC.
`assemblies`	`const EipAssembly-Config *`	Assembly array.
`assembly_count`	`uint8_t`	0 keeps the test set (table 4.2).
`timeout_floor_ms`	`uint32_t`	Minimum inactivity timeout. 0 uses RPI × multiplier, but never less than two clock ticks.
`netif_index`	`uint8_t`	The port interface that the adapter reports. 0 is the first.
`link_info`	function pointer	Optional. Link state, speed, and duplex.
`apply_interface`	function pointer	Optional. Applies new TCP/IP settings. 0: read-only.
`reset`	function pointer	Optional. Identity Reset. 0 refuses the service.
`callback_ctx`	`void *`	First argument of each callback.
`domain`	`const char *`	Initial domain name for TCP/IP attribute 5. Can be 0.

Field	Type	Meaning
`allow_change`	function pointer	Optional. Permits or refuses an interface change (`EIP_CHANGE_INTERFACE`) or a reset (`EIP_CHANGE_RESET`).

Table B.8. `EipAdapterView`: adapter snapshot

Field	Type	Meaning
`running`	`uint8_t`	The adapter has started.
`listening`	`uint8_t`	Its sockets are open.
`vendor_id` … `revision_minor`	various	Identity values.
`serial_number`	`uint32_t`	Serial number.
`name`	`char[32]`	Product name.
`ip`	`uint8_t[4]`	Reported address.
`io_open`	`uint8_t`	A Class 1 connection is open.
`run`	`uint8_t`	The run/idle header says run, or the connection is modeless.
`multicast`	`uint8_t`	T→O goes to a multicast group.
`o2t_instance`, `t2o_instance`	`uint16_t`	Assemblies of the open connection.
`rpi_us`	`uint32_t`	T→O interval of the open connection.
`timeout_ms`	`uint32_t`	Inactivity timeout of the open connection.
`rx_packets`, `tx_packets`	`uint32_t`	Class 1 packets in and out.
`opens`	`uint32_t`	Successful Forward_Opens.
`timeouts`	`uint32_t`	Connections closed by the watchdog.
`rx_dropped_seq`	`uint32_t`	Duplicate or old sequence number.
`rx_dropped_source`	`uint32_t`	O→T from an unexpected address.
`rx_dropped_bad`	`uint32_t`	Wrong connection ID, too short, or malformed.
`rx_idle`	`uint32_t`	Accepted packets in idle (data set to zero).
`udp_replies_dropped`	`uint32_t`	UDP 44818 requests not answered (rate limit).
`originator_ip`	`uint8_t[4]`	T→O destination: the originator or the multicast group.
`peer_ip`	`uint8_t[4]`	The originator itself.
`tcp_clients`	`uint8_t`	Open explicit-message sessions.
`assembly_count`	`uint16_t`	Number of assemblies.
`assemblies`	array of `EipAssembly-View`	Instance, direction, size, and name of each.

Table B.9. `EipInterfaceConfig`: settable TCP/IP values

Field	Type	Meaning
`control`	`uint8_t`	Attribute 3: 0 static, 2 DHCP.
`ip`, `netmask`, `gateway`	`uint8_t[4]`	Attribute 5.
`dns1`, `dns2`	`uint8_t[4]`	Attribute 5.
`domain`	`char[49]`	Attribute 5.
`host_name`	`char[65]`	Attribute 6.

Table B.10. `EipNetifInfo`: interface data from a port

Field	Type	Meaning
`ip`, `netmask`, `gateway`	`uint8_t[4]`	Address values.
`mac`	`uint8_t[6]`	MAC address. Sets the default serial number.
`dns1`, `dns2`	`uint8_t[4]`	DNS servers.
`dhcp`	`uint8_t`	1 if the address came from DHCP.
`link_known`	`uint8_t`	1 if the link fields are valid.
`link_up`, `full_duplex`, `autonegotiated`	`uint8_t`	Link state.
`speed_mbps`	`uint32_t`	Link speed.
`host_name`	`char[65]`	Host name.

Appendix C

Codes and Constants

C.1 Return Codes

Table C.1. QuickEIP return codes

Code	Value	Meaning
`EIP_OK`	0	Success.
`EIP_ERR_ARG`	−1	Bad argument.
`EIP_ERR_CONNECT`	−2	TCP connect failed.
`EIP_ERR_IO`	−3	A socket failed or the peer closed.
`EIP_ERR_TIMEOUT`	−4	No reply in the timeout.
`EIP_ERR_PROTOCOL`	−5	Malformed or unexpected reply, or an encapsulation status that is not 0.
`EIP_ERR_CIP`	−6	The device answered with an error. See `eip_last_cip_status`.
`EIP_ERR_TOO_BIG`	−7	Does not fit the buffer of the caller or a protocol limit.
`EIP_ERR_PATH`	−8	Bad tag or path text, or no such assembly instance.
`EIP_ERR_STATE`	−9	Wrong state: session not open, or adapter not started or started two times.

C.2 CIP Status Codes

Table C.2 lists the CIP status codes that this manual refers to. The ODVA specifications give the complete list.[1]

C.3 CIP Data Types

C.4 Ports and Other Constants

C.5 Identity Status Word of the Adapter

[1] ODVA, *The CIP Networks Library*, vol. 1, *Common Industrial Protocol*, app. B, Status Codes.

Table C.2. CIP status codes used in this manual

General / extended	Meaning in QuickEIP
`0x01` / `0x0106`	Another originator owns the connection.
`0x01` / `0x0113`	Another originator owns the connection (request without O→T).
`0x01` / `0x0114`–`0x0116`	The electronic key does not agree.
`0x01` / `0x0126`	Config data size does not agree with the config assembly.
`0x01` / `0x0127`	O→T size is wrong.
`0x01` / `0x0128`	T→O size is wrong.
`0x01` / `0x0129`	Config instance does not exist.
`0x01` / `0x012A`	Consumed instance does not exist.
`0x01` / `0x012B`	Produced instance does not exist.
`0x01` / `0x0204`	Routed request: the slot did not answer in time.
`0x04`	Path segment error: malformed path.
`0x05`	Path destination unknown: tag or object not found.
`0x06`	Partial transfer. `eip_cip` returns `EIP_OK`.
`0x10`	Device state conflict. Usual value from `allow_change`.
`0x16`	Object does not exist, for example an unknown or implicit-only assembly.

Table C.3. CIP elementary data types

Constant	Code	Type	Bytes
`EIP_BOOL`	`0x00C1`	Boolean	1
`EIP_SINT`	`0x00C2`	Signed 8-bit	1
`EIP_INT`	`0x00C3`	Signed 16-bit	2
`EIP_DINT`	`0x00C4`	Signed 32-bit	4
`EIP_LINT`	`0x00C5`	Signed 64-bit	8
`EIP_USINT`	`0x00C6`	Unsigned 8-bit	1
`EIP_UINT`	`0x00C7`	Unsigned 16-bit	2
`EIP_UDINT`	`0x00C8`	Unsigned 32-bit	4
`EIP_ULINT`	`0x00C9`	Unsigned 64-bit	8
`EIP_REAL`	`0x00CA`	IEEE 754 single	4
`EIP_LREAL`	`0x00CB`	IEEE 754 double	8

Table C.4. Network ports and constants

Constant	Value	Use
`EIP_TCP_PORT`	44818	Encapsulation (TCP and UDP).
`EIP_UDP_PORT`	2222	Class 1 I/O on the adapter.
`EIP_SCAN_PORT`	44819	Local UDP port of the broadcast scan.
—	2223–2231	Local UDP ports of a scanner Class 1 connection.
`EIP_SOCK_INVALID`	−1	No socket.
`EIP_KIND_ASSEMBLY`	0	Generic EtherNet/IP device.
`EIP_KIND_COMPACTLOGIX`	1	Rockwell 1768, 1769, 5069, or a CompactLogix name.
`EIP_KIND_MICRO800`	2	Rockwell Micro800.
`EIP_CHANGE_INTERFACE`	1	`allow_change`: a TCP/IP Interface Set.
`EIP_CHANGE_RESET`	2	`allow_change`: an Identity Reset.

Table C.5. Identity status of the adapter

Value	State
`0x0034`	Configured, no Class 1 connection
`0x0065`	Class 1 connection in run
`0x0075`	Class 1 connection in idle

Appendix D

Tools, Samples, and Migration

D.1 eiptool

`eiptool` is a command-line scanner and test adapter. It uses the host port (POSIX or Windows). Its source is `examples/posix_eiptool/eiptool.c`, and it uses most of the scanner API.

Table D.1. eiptool commands

Command	Action
`eiptool scan`	Broadcast ListIdentity and list the devices.
`eiptool identity <ip>`	Read the identity of one device.
`eiptool tags <ip> [slot]`	List the tags of a controller.
`eiptool read <ip> <tag> [slot]`	Read a tag.
`eiptool write <ip> <tag> <type> <value> [slot]`	Write a tag. `type` is `dint`, `int`, `sint`, `real`, or `bool`.
`eiptool get <ip> <class> <instance> <attribute> [slot]`	Get_Attribute_Single.
`eiptool adapter`	Run the built-in test adapter until you stop it.

These rules apply:

- `slot` is −1 by default.
- Numbers can be decimal or hexadecimal with `0x`.
- Set `EIP_NETIF=`*name* to select the interface that the adapter reports and that the scan uses first.
- Explicit messages operate on `127.0.0.1`. Class 1 needs the address that the adapter reports.

```
./build/eiptool adapter &
./build/eiptool get 127.0.0.1 4 101 3    # test assembly: 11 22 33 44 55 66 77 88
./build/eiptool get 192.168.1.10 0x01 1 7  # class instance attribute
./build/eiptool write 192.168.1.10 MyDint dint 42
```

D.2 Examples in the Repository

D.3 Sample EDS File

These are the device, assembly, and connection sections of `examples/eds/QuickEIP_Adapter.eds`. They agree with the assemblies in section 4.2. Vendor code 65000 is a placeholder.

```
[Device]
        VendCode = 65000;
        VendName = "QuickEIP Sample";
        ProdType = 43;
        ProdTypeStr = "Generic Device";
```

Table D.2. Example programs

Directory	Shows
`examples/netburner_adapter`	An adapter on NetBurner, and an optional scanner tag read.
`examples/esp_idf_adapter`	An adapter as an ESP-IDF project.
`examples/clearcore_adapter`	A poll-mode adapter that controls ClearCore I/O.
`examples/posix_eiptool`	`eiptool`, with most of the scanner API.
`examples/eds`	A sample EDS file that agrees with the examples.
`examples/netburner_webui`	A browser test page. Application code, not part of the stack.

```
        ProdCode = 1;
        MajRev = 1;
        MinRev = 1;
        ProdName = "My Device";
        Catalog = "My Device";

[Assembly]
        Revision = 2;
        Object_Name = "Assembly Object";
        Object_Class_Code = 0x04;
        Number_Of_Static_Instances = 3;
        Assem100 =
                "Inputs",               $ device -> PLC (EIP_ASM_PRODUCED)
                "20 04 24 64 30 03",    $ path
                32,                     $ size in bytes
                0x0000,                 $ descriptor
                ,,                      $ reserved
                256,;                   $ member size in bits
        Assem150 =
                "Outputs",              $ PLC -> device (EIP_ASM_CONSUMED)
                "20 04 24 96 30 03",
                32,
                0x0001,
                ,,
                256,;
        Assem151 =
                "Config",               $ configuration, no data
                "20 04 24 97 30 03",
                0,
                0x0000,
                ,,
                0,Param2;

[Connection Manager]
        Connection1 =
                0x04010002,             $ class 1, cyclic, exclusive owner
                0x44640405,             $ O->T p2p, fixed, run/idle; T->O mcast or p2p
                Param1,32,Assem150,     $ O->T RPI, size, format
                Param1,32,Assem100,     $ T->O RPI, size, format
                ,,                      $ config part 1 (none)
                0,Param2,               $ config part 2: size, format
                "Exclusive Owner",      $ name
                "",                     $ help
                "20 04 24 97 2C 96 2C 64";  $ config 151, consume 150, produce 100
```

```
[Capacity]
        MaxIOConnections = 1;
        MaxMsgConnections = 2;
        TSpec1 = Rx, 32, 100;
        TSpec2 = Tx, 32, 100;
```

D.4 Migration from QuickEIP 1.x

QuickEIP 2.0 replaces the NetBurner-only version 1.x with a portable library. The adapter has these new functions: a configurable identity and assemblies, a shared assembly data pool, multicast T→O, the TCP/IP Interface and Ethernet Link objects, a settable IP configuration, Identity Reset, and checks of the input. The sizes are now in `eip_config.h`.

Procedure: Change a 1.x NetBurner build to 2.0

1. Remove the old source lines for `eip_session.cpp`, `eip_io.cpp`, and `eip_adapter_task.cpp` from the makefile.
2. Add the source list of section 7.1.
3. Change all timeouts from ticks to milliseconds (table D.3).
4. Change `eip_adapter_start()` to `eip_adapter_start(0)`, or give a configuration.
5. If you need the 1.x timeout, set `cfg.timeout_floor_ms = 2000`.
6. If you use the web UI, add the files from `examples/netburner_webui/`.

CAUTION

All timeouts are now in milliseconds. A value of `TICKS_PER_SECOND` (20) that meant 1 s in version 1.x now means 20 ms. Examine each timeout argument.

Table D.3. API changes from version 1.x

1.x	2.0	Notes
`EipOpenConfig.timeout_ticks`	`timeout_ms`	`TICKS_PER_SECOND * 5` becomes `5000`.
`eip_list_identity(ip, id, ticks)`	`(ip, id, ms)`	
`eip_scan_devices(d, n, ticks)`	`(d, n, ms)`	0 still selects the default (3 s).
`eip_io_consume(io, d, n, ticks)`	`(io, d, n, ms)`	
`session.tcp_fd > 0`	`eip_session_is_open(&session)`	The field is now `sock`, and 0 is a valid handle.
`io.udp_fd`	`io.sock`	
`EipSession.timeout_ticks`	`timeout_ms`	

1.x	2.0	Notes
`eip_adapter_start(void)`	`eip_adapter_start(const EipAdapterConfig *)`	Give 0 for the old behavior.
`EIP_MAX_PACKET` (enum)	Macro in `eip_config.h`	Same value. You can now change it.
`EIP_ADAPTER_ASSEMBLIES` (4)	`EIP_ADAPTER_MAX_ASSEMBLIES` (8)	The old name is still an alias.
`src/eip_port.h`	`src/eip_internal.h`	Internal header, new name.
`src/eip_adapter.h`	`include/eip_adapter.h`	Now public (the sans-I/O core).
`EipAdapterView`	More fields	Added `ip`, `multicast`, `rpi_us`, `timeout_ms`, `opens`, `timeouts`, `rx_dropped_*`, `rx_idle`, `udp_replies_dropped`, `tcp_clients`.

Changes of behavior:

- The inactivity timeout now comes from the O→T RPI (equation 4.3). Version 1.x used the T→O interval and a 2 s minimum.
- The Identity status now shows "configured" (table C.5).
- An unknown assembly instance answers CIP `0x16`, not `0x05`.
- Consumed data goes to all zeros on idle or timeout, as in version 1.x. On idle, the adapter clears only the consumed point of the connection.
- The adapter accepts up to four TCP clients, not two. When all slots are busy, it drops a session that has been silent for 60 s.
- The 16-bit sequence count is still used for originators that send no Sequenced Address item. A repeated count keeps the connection open but does not replace the data. An older count is dropped.

Glossary

Adapter
A device that a PLC sees as an I/O device. It is the target of a Class 1 connection. Also called the target.

Assembly
A block of data with an instance number in the Assembly object (`0x04`). Assemblies are consumed, produced, or config.

Class 1 connection
An implicit (I/O) connection that sends assembly data on UDP 2222 at a fixed interval. A Forward_Open over TCP opens it.

CIP
Common Industrial Protocol. The object model and services that EtherNet/IP carries. CIP is a trademark of ODVA.

Consumed assembly
An assembly that the adapter receives from the PLC (O→T).

EDS file
Electronic Data Sheet. A text file that tells PLC software the identity, assemblies, and connections of a device.

Electronic key
Identity values in a Forward_Open that the target compares with its own values.

Encapsulation
The EtherNet/IP header around each CIP message on TCP or UDP 44818.

Explicit message
A request and reply message, for example Get_Attribute_Single. QuickEIP sends them unconnected, with SendRRData.

Forward_Open
The Connection Manager service that opens a connection. Forward_Close closes it.

HAL
Hardware abstraction layer. In QuickEIP, the 19 port functions in `eip_hal.h`.

Inactivity timeout
The time after which the target closes a connection if no valid O→T packet arrives. See equation 4.3.

ListIdentity
An encapsulation command that asks a device for its identity. A scan sends it as a broadcast.

Modeless
A connection direction without a run/idle header.

Multicast
A T→O connection that sends to an IP group address, not to one host.

O→T
Originator to target: the direction from the PLC or scanner to the adapter.

Originator
The device that opens a connection, usually a PLC or a scanner.

PLC
Programmable logic controller.

Poll mode
A port without tasks. The application calls `eip_adapter_poll` from its main loop.

Port
The platform-specific code that implements `eip_hal.h`.

Produced assembly
An assembly that the adapter sends to the PLC (T→O).

RPI
Requested packet interval. The time between two Class 1 packets in one direction.

Run/idle header
A 4-byte header before O→T data. Bit 0 is 1 in run and 0 in idle (Program mode).

Sans-I/O
A design in which the protocol code does not use sockets or a clock. The caller supplies packets and the time.

Scanner
A device that opens sessions and connections to other devices. In QuickEIP, the originator role.

Session
A TCP connection on port 44818 with a registered session handle.

Slot
The position of a module in a ControlLogix chassis. The scanner uses it to route a request across the backplane.

T→O
Target to originator: the direction from the adapter to the PLC or scanner.

Target
The device that accepts a connection. In QuickEIP, the adapter role.

Unconnected Send
A Connection Manager service that routes a message through a router to another port or slot.

Bibliography

ASD (AeroSpace, Security and Defence Industries Association of Europe). *ASD-STE100: Simplified Technical English.* Issue 8. Brussels: ASD, 2021.

The Chicago Manual of Style. 18th ed. Chicago: University of Chicago Press, 2024.

Espressif Systems. *ESP-IDF Programming Guide.* Version 5.5. Shanghai: Espressif Systems. `https://docs.espressif.com/projects/esp-idf/`.

ODVA. *The CIP Networks Library.* Vol. 1, *Common Industrial Protocol (CIP).* Ann Arbor, MI: ODVA.

———. *The CIP Networks Library.* Vol. 2, *EtherNet/IP Adaptation of CIP.* Ann Arbor, MI: ODVA.

Sweeney, Adam G. "Integrating QuickEIP." `docs/INTEGRATION.md` in *QuickEIP*, version 2.0.0. Source code repository, 2026.

———. "Porting QuickEIP to a New Platform." `docs/PORTING.md` in *QuickEIP*, version 2.0.0. Source code repository, 2026.

———. *QuickEIP: A Small EtherNet/IP Stack for Embedded Targets.* Version 2.0.0. Source code repository, MIT License, 2026.

Function Index

This index gives the page of the reference entry for each public function. Appendix A contains the entries.

eip_adapter_default_server 59
eip_adapter_poll 57
eip_adapter_read 57
eip_adapter_snapshot 57
eip_adapter_start 57
eip_adapter_write 58
eip_cip 55
eip_clearcore_set_host_name 59
eip_device_kind 55
eip_esp_set_netif 59
eip_get_attribute 56
eip_hal_adapter_start_task 61
eip_hal_clock_resolution_ms 59
eip_hal_lock 60
eip_hal_millis 59
eip_hal_netif_get 60
eip_hal_tcp_accept 60
eip_hal_tcp_close 61
eip_hal_tcp_connect 60
eip_hal_tcp_listen 60
eip_hal_tcp_local_ip 61
eip_hal_tcp_recv 60
eip_hal_tcp_send 60
eip_hal_udp_close 61
eip_hal_udp_open 61
eip_hal_udp_recvfrom 61
eip_hal_udp_sendto 61
eip_hal_unlock 60
eip_hal_wait 61
eip_hal_yield 60
eip_io_close 57
eip_io_consume 57
eip_io_open 56
eip_io_produce 56
eip_last_cip_extended 54
eip_last_cip_status 54
eip_last_encap_status 54
eip_list_identity 55
eip_list_symbols 55
eip_load_le16 53
eip_load_le32 53
eip_parse_ipv4 53
eip_scan_devices 55
eip_server_close 58
eip_server_init 58
eip_server_poll 58
eip_server_read 58
eip_server_service 58
eip_server_snapshot 58
eip_server_write 59
eip_session_close 54
eip_session_is_open 54
eip_session_open 54
eip_session_set_slot 54
eip_set_attribute 56
eip_status_text 53
eip_store_le16 53
eip_store_le32 53
eip_tag_read 56
eip_tag_write 56